SoK: Instruction Set Extensions for Cryptographers

Hao Cheng[1,2,3], Johann Großschädl[4], Ben Marshall[5], Daniel Page[6] and Markku-Juhani O. Saarinen[7]

[1] School of Cyber Science and Technology, Shandong University, Qingdao, China.
[2] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China.
[3] Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao, China.
hao.cheng@sdu.edu.cn
[4] DCS and SnT, University of Luxembourg, Esch-sur-Alzette, Luxembourg.
johann.groszschaedl@uni.lu
[5] PQShield Ltd, Oxford, UK.
ben.marshall@pqshield.com
[6] School of Computer Science, University of Bristol, Bristol, UK.
daniel.page@bristol.ac.uk
[7] SoC Hub Research Centre, Tampere University, Tampere, Finland.
markku-juhani.saarinen@tuni.fi

Abstract. Framed within the general context of cyber-security, standard cryptographic constructions often represent an enabling technology for associated solutions. Alongside or in combination with their design, therefore, the implementation of such constructions is an important challenge: beyond delivery of usable artefacts, implementation can impact many quality metrics (such as efficiency and security) which determine fitness-for-purpose. In at least some use-cases, software-based implementation of cryptography may be attractive or even mandated. Such an implementation is heavily influenced both by 1) the Instruction Set Architecture (ISA) it is expressed using, and 2) the micro-architecture it is executed using. For example, the extent to which a general-purpose ISA can support more domain-specific requirements of a cryptographic construction will influence how the latter is mapped to the former (i.e., which implementation techniques are viable) *and* behavioural properties of doing so (e.g., the execution latency stemming from use of an implementation technique). This paper acts to systematise the topic of cryptographic Instruction Set Extensions (ISEs), which represent one approach to provision of a platform where such support is more explicit and extensive.

Keywords: ISA, ISE, cryptographic engineering

1 Introduction

1.1 The micro-processor customisation design space

ISAs. The concept of an ISA is fundamentally important within the context of computer systems. Acting as an interface between hardware, i.e., some compliant micro-architectural implementation, and software executed by it, the ISA will usually define components such as 1) the accessible state, including general- and special-purpose registers and memory, 2) the encoding and semantics of instructions that act on said state, and 3) an execution

model for said instructions. The design of an ISA is, to some extent, a creative process driven by technical principles which inform design decisions and trade-offs. However, the large design space, potential lifespan, and implications that features in an ISA have for hardware and software, demand the process is *also* driven by robust empirical evidence. This is often achieved through characterisation of software workloads, in order to motivate and evaluate features in the design. To appeal to the largest possible market such workloads are usually of a *general*-purpose composition (e.g., representative benchmark kernels and suites thereof, or complete applications), with evaluation with respect to pertinent quality metrics (e.g., instruction throughput) usually focusing on optimisation for the average-case. Although this approach is rational, it *may* be viewed as conservative in the sense it neglects opportunities related to *special*-purpose or *domain-specific* workloads.

The ISA is, and arguably [DB18] should remain an interface. This approach allows specialisation of a micro-architecture to satisfy any pertinent quality metric, market, or use-case, and thus, to some extent at least, workload. Beyond this fact, however, micro-processor customisation [IL07] captures broader specialisation of *both* the implementation *and* interface for a given domain. In short, a customisable micro-processor may be either parameterisable, in the sense that a template base ISA and micro-architecture are instantiated based on user-selected options, and/or extensible, in the sense that a base ISA and micro-architecture can be extended with user-defined functionality.

ISEs. ISEs have become a popular extensibility mechanism. In concept, the ISE design process typically mirrors that for ISAs apart from a focus on some specific domain. That is, one would 1) perform workload characterisation to identify functionality that could yield an improvement with respect to pertinent quality metrics (over use of the base ISA alone), 2) extend the base ISA with suitable state and instructions, thereby exposing them to ISE-aware software, then, finally, 3) implement suitable additions or alterations to a base micro-architecture, thereby allowing said software to be executed.

Other implementation options for a given workload include at least software-only (i.e., using the base ISA alone) and hardware-only (i.e., using a dedicated IP core) extremes. ISEs are attractive specifically because they represent a hybrid that sits between such extremes, suggesting they inherit characteristics of both; in a general sense, therefore, they relate to the field of hardware/software co-design [DEWW01]. Whether or not they are the *right* option is heavily dependent on the context, but, for example, an ISE-supported solution will often be more compact and performant than a software-only option, while also being more flexible and more efficient in terms of performance improvement per additional logic gate than a hardware-only option. Such characteristics can be important for *both* high-end, performance-oriented *and* low-end, constrained platforms.

Although isolated examples[1] existed previously, media processing, e.g., decoding image and video formats such as JPEG and MPEG, was (arguably) the domain which popularised ISE-supported solutions more broadly. Market demand for efficiency in relation to such workloads led to development of ISEs [SS05] for most contemporary base ISAs, including AMD 3DNow! [OFW99], DEC Motion Video Instructions (MVI) [CCM97], Hewlett Packard Multimedia Acceleration eXtensions (MAX and MAX-2) [Lee95, Lee96, LH96], Intel MultiMedia eXtensions (MMX) [PW96], Motorola Altivec [DDHS00], MIPS Digital Media eXtension (MDMX) and MIPS-3D, and Sun Visual Instruction Set (VIS) [KMPZ95].

Subsequent examples highlight a subtlety, in the sense that the definition of an *extension of* versus an *addition to* the base ISA is imprecise: several of the ISEs listed above would arguably fall into the latter category. For example, what was initially Intel MMX has evolved into several generations of Streaming SIMD Extensions (SSE) [TH99] and then

[1] One such example is support for computation of population count (i.e., Hamming weight), exemplified by the x86 `popcnt` [Int22b, Pages 4-399–4-400] instruction. Motivation for what is a fairly niche operation has an interesting (and in part alleged) history: see, e.g., https://groups.google.com/g/comp.arch/c/UXEi7G6WHuU/m/Z2z7fC7Xhr8J.

Advanced Vector Extensions (AVX). Such designs addressed market demand for efficient numerical computation, offering support via a vector[2] versus scalar programming model; Intel-based micro-architectures rarely *remove* features, meaning the ISEs effectively act as additions to the base ISA therefore. In contrast, ISAs such as RISC-V emphasise modularity and extensibility as first-class, by-design goals (see, e.g., [RIS24, Section 37]). RV32I [RIS24, Section 2], for example, defines a minimal base ISA plus a suite of (orthogonal) extensions which can be categorised as 1) standard extensions curated and ratified by RISC-V International; selected examples aim to support additional functionality (e.g., floating-point, via the standard F [RIS24, Section 20] and D [RIS24, Section 21] extensions), or satisfy specific optimisation goals (e.g., code density, via the standard C [RIS24, Section 26] extension), plus 2) non-standard (or custom) extensions which are user-defined. Support for so-called "custom instructions" [CP20] within ARMv8-M could be viewed as having a similar, although more limited remit.

ASIPs. The difference between extension of a base ISA which does or does not have by-design support for extensibility is important. For example, where such support *is* evident, a vendor can decide whether or not a given a set of orthogonal domains are relevant to their intended market, then either include or exclude ISEs as need be.

This is taken to an extreme by the concept of an Application Specific Instruction Processor (ASIP), definitions of which can vary but usually include three features. First, both the base ISA and micro-architecture are explicitly designed to support customisation. Second, tasks relating to customisation of these artefacts are often supported by an integrated suite of dedicated, (semi-)automatic tooling. Tensilica Xtensa [Gon00] is an exemplar of both points. The platform includes an 80-instruction base ISA and associated base micro-architecture which are both parameterisable (e.g., with respect to endianness, register file size, cache geometry, bus width, etc.), and extensible (via extensions to the base ISA, i.e., ISEs) by-design; the latter is supported by a proprietary language, namely Tensilica Instruction Extension (TIE), and some associated tooling. Moreover, third, trade-offs for an ASIP can be aggressive with respect to specialisation. Put simply, an ASIP might *only* be used within a specific domain or for a specific task. This contrasts with a non-ASIP, which must support general-purpose workloads and use, e.g., ISEs to support special-purpose niches.

1.2 Toward principled, effective cryptographic ISE development

Cryptographic ISEs. The domain of cryptography shares various high-level characteristics with that of media processing. For example, cryptographic workloads typically 1) involve computationally intensive, somewhat niche functionality, 2) need to satisfy a range of efficiency-related quality metrics such as throughput, latency, memory footprint, and power and energy consumption, but, at the same time, *also* 3) form a central target in what is a complex, evolving attack surface. This latter fact demands (see, e.g., [RKL+04, RRKH04, BMT16]) security be viewed an important, additional quality metric. A rich body of literature, capturing the field of cryptographic engineering, has explored techniques which address associated implementation challenges (including those relating to micro-processor design specifically: see, e.g., [Lee03]); this forms a significant design space of options.

Nahum et al. [NOOS95] argue that, although cryptographic hardware can be a viable option, various factors such as flexibility (or agility) mean "*we need cryptographic software*". Their work is among, if not the first to propose an ISE-supported solution as a means of addressing that need in balance with other metrics. This suggests the field of cryptographic

[2]Although the term Single Instruction Multiple Data (SIMD) is often used to describe such support (even within names of concerete ISEs), the programming model is more precisely described as "packed SIMD" or SIMD Within A Register (SWAR).

ISEs [BGM09, HV11, RI16] spans at least a 25 year period, with support for Advanced Encryption Standard (AES) [NIST01] an important exemplar. For example, one can readily identify a significant body of literature [NIK04, TG05, TGS05, BBFR06, MD07, TG06, TG07a, TG07b, Elb07, Elb08, BBGR09, SCS+09, JSG10, BOS11, APRJ11, BEM+15, EMOC20, Saa20, IPK+24, ISP24, Saa25, NPL+25], patents [GFG], and support in deployed base ISAs: instances include at least x86 [Int22a, Section 12.13] (see also [Gue09, DGK19]), POWER [Pow18, Section 6.11.1], ARMv8-A [Arm20, Section A2.3], SPARC [SPA16, Sections 7.3+7.4], and RISC-V [RIS24, Sections 32.3.4, 32.3.5, and 33.2.5] (see also [MPP20, MNP+21]).

Scope and contributions. Nahum et al. [NOOS95] pondered whether, in 1995, it was "*practical to add instructions to a [RISC] processor*"; they identify both technological and economic dimensions to this question. Whether or not the answer was positive then, we argue that two features of the contemporary technology landscape mean it certainly *is* positive now. First, open ISAs such as RISC-V [Wat16], and associated ecosystems (including open-source implementations, tooling, and crucially, community) have matured to the point of being competitive. Hill et al. [HCP+16] explore advantages and disadvantages of this approach, but, fundamentally, it has enhanced accessibility and so innovation where it would previously, in a proprietary context, be far more difficult. Second, concrete realisation of such implementations is now viable with relatively short, relatively low-cost design cycles for a given ISE. At least two facts evidence this claim:

1. the availability and feature-sets of reconfigurable fabrics: this fact leads to viability of customisable FPGA-based soft *and* hard (e.g., supported by eFPGAs and similar technologies) implementations, and
2. an evolution of business models: this fact leads to viability of customisable ASIC-based hard micro-processor implementations (e.g., as the result of new vendors and initiatives which have lowered the barrier to entry versus traditional fabrication).

So, based on the argument that cryptographic ISEs are both practical and effective (cf. [FLO18]), and mirroring [IL07, Section 1.4] for example, we suggest that the field is entering a "golden age" in which maximising the quality of associated work is an important challenge. This paper attempts to assist in doing so, by representing a comprehensive Systematization of Knowledge (SoK) for the topic of cryptographic ISEs. We structure the paper in alignment with the following high-level contributions:

1. In Section 2, we present a conceptual and terminological framework for the topic of cryptographic ISEs.
2. In Section 3, we use the structure provided by Section 2 to present a detailed taxonomy of existing cryptographic ISEs.
3. In Section 4, we use (positive and negative) evidence highlighted in Section 3 to present a structured cryptographic ISE development process; said process captures both generic principles and techniques and technologies, and spans steps including design, implementation, verification, and evaluation.
4. In Section 5 we adopt a more future-facing perspective: based on coverage in Section 3 and Section 4, for example, we identify and discuss related concepts, emerging trends, and open challenges (or opportunities) related to the topic of cryptographic ISEs.

The introductory paragraph of each Section motivates and explains the associated contribution, in terms of both the role content has in systematising the topic (i.e., why it is necessary) and to what extent the content is either novel or extends the existing state-of-the-art (e.g., existing surveys such as [BGM09, RI16]).

We focus on two overarching aims throughout. First, high quality cryptographic engineering is inherently interdisciplinary. In line with advice from Paar [Paa02, Point 5]

that we "*educate ourselves about other fields*" and also "*entice people from other disciplines to work on problems in applied cryptography*", we therefore aim to consolidate experience around cryptographic ISEs and so increase future understanding and accessibility of the field. The paper title is specifically motivated by this point, and so is written in a similar spirit as, e.g., [GPS08, BGHZ11]. Second, consider that AES[3] was instrumental in popularising a model for standardisation processes where "*algorithm and implementation characteristics*" play a role in evaluation of candidates. Although seldom *explicitly* ruled out, many such processes fix (a set of) non-customisable evaluation platforms which *implicitly* ignore or undervalue the role ISEs can and eventually *do* play. We argue this gap is problematic with respect to comprehensive evaluation, so aim to help shift best-practice to include consideration of ISEs. Finally, note that we often articulate points by using existing ISE designs as examples. However, in doing so, we focus purely on a fact- rather than opinion-based analysis (and so, e.g., specifically avoid critiquing such designs). This is rationalised by the difficulty of identifying a "best" solution: an overt aim of Section 4 is to highlight that only by considering trade-offs between constraints, metrics, etc. can one identify a "best-fit" solution in a given context. For example, the AES New Instructions (AES-NI) [Gue09, Gue10] design might have been the best-fit for x86; the fact it was not for RISC-V is not definitively a positive or a negative in relation to the design itself.

1.3 Notation

Let $x_{(b)}$ denote x expressed in radix- or base-b. If the base is omitted, it is safe to assume use of decimal (i.e., that $b = 10$). Let $x \leftarrow y$ denote assignment of y to x. Let $\neg$, $\wedge$, $\vee$, and $\oplus$ denote the Boolean NOT, AND, (inclusive) OR, and (exclusive OR, or) XOR operators respectively, and $x \ll y$ and $x \lll y$ (resp. $x \gg y$ and $x \ggg y$) denote left-shift and left-rotate (resp. right-shift and right-rotate) of x by y bits.

Let $\mathsf{MEM}[i]^b$ denote a b-byte access to some byte-addressable memory, using the effective address i; where $b = 1$, the access granularity may be omitted. Let $\mathsf{GPR}[i]$, for $0 \leq i < r$, denote the i-th, w-bit entry in the r-entry general-purpose register file.

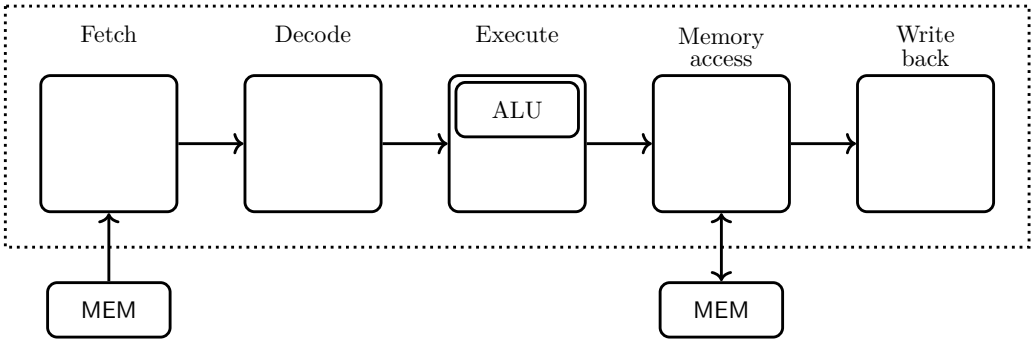
2 A conceptual and terminological framework

In this Section, we present a conceptual and terminological framework for the topic of cryptographic ISEs. We claim that doing so represents a vital pre-requisite for all subsequent Sections, e.g., due to the strongly inter-disciplinary nature of this topic: ISE design involves a wide range of diverse technical concepts spanning both hardware and software, *and* a wide range of stakeholders including 1) policy and decision makers, who fix constraints and select between options, e.g., based on pertinent use-cases, 2) cryptographers, who focus on design of the underlying constructions, 3) digital design engineers (cf. [BMT16]), who focus on hardware designs and implementations, 4) cryptographic engineers, who focus on software designs and implementations. As such, definitions within such a framework are required to enable clear, precise description of a given ISE design and associated context, and any (comparative) evaluation of (or relative to) it.

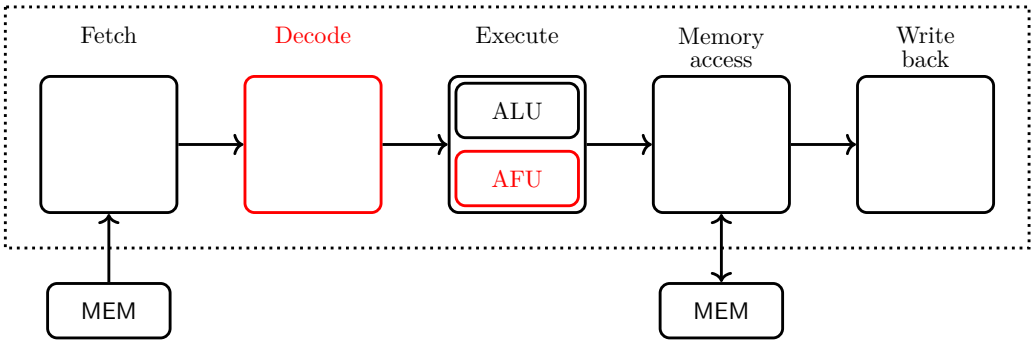
Concept 1. *An ISE comprises 1) a specification of an* **interface** *between hardware and software, plus 2) a hardware-based* **implementation** *within some micro-architecture.*

Concept 2. *The terms* **base ISA** *and* **extended ISA** *(resp.* **base micro-architecture** *and* **extended micro-architecture***) are used to refer to the ISA (resp. micro-architecture) excluding and including the ISE interface (resp. implementation) respectively.*

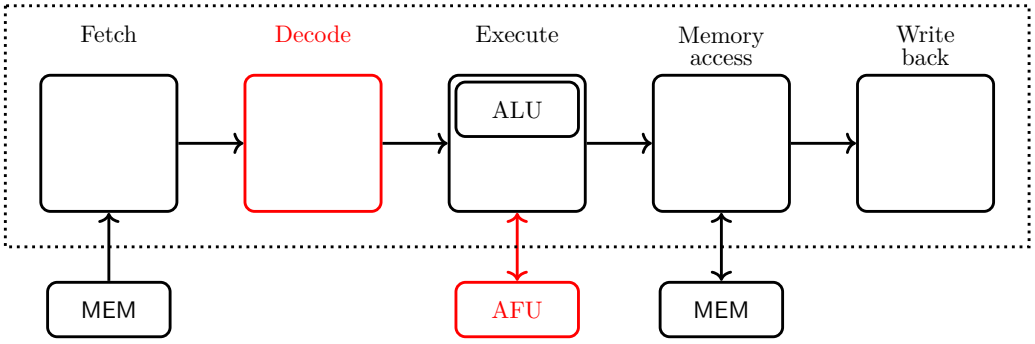
[3] See, e.g., https://www.govinfo.gov/content/pkg/FR-1997-09-12/pdf/97-24214.pdf



(a) The base core.



(b) The base core plus tightly-integrated ISE.



(c) The base core plus loosely-integrated ISE.

Figure 1: An high-level, illustrative example focused on (the 5-state pipeline constituting the micro-architecture of) a base core, and a hypothetical but typical computation-oriented ISE implementation within it. Note that 1) a dashed box highlights intra- versus extra-core components, 2) arrows between components imply some form of interface (and perhaps protocol to manage it), and 3) components coloured red impacted by the ISE implementation.

As with an ISA, the ISE interface allows diversity with respect to the associated implementation. Versus an ISA, however, it is common to consider the ISE interface and implementation simultaneously; the term ISE is often overloaded, therefore, to mean either the interface and/or the implementation depending on the context.

Concept 3. *A* **bespoke ISA** *is explicitly domain-specific by nature, and so distinct from some* **generic ISA** *being extended by a domain-specific ISE.*

There is clearly overlap between the two approaches, and, up to a point, the end result could be deemed similar. However, we distinguish between them in part to limit our scope through a focus on the latter: doing so renders some examples of the former (e.g., ASIP designs such as CryptoManiac [WWA01], Cryptonite [BHO04], MCCP [GBG+11], CIARP [NRE+12], Cryptoraptor [SC14], CRISC [DR14], CDSP [YHMH19], plus related tools such as [MA07], and work such as [GBG+03, Fou07, KSG08, YYDZ08, KZS+09, BSP13, YSKG14, SEM17]) out of scope. Beyond this fact, however, focusing on the extension of a generic, ideally well established base ISA affords (at least) two advantages. First, the use of a generic base ISA makes it possible to leverage both experience and infrastructure (e.g., libraries, tool-chains, etc.) which already exist; in essence, it offers a lower barrier to entry. Second, the opportunity to design a bespoke ISA is a rare occurrence. Focusing on a generic base ISA therefore maximises the potential for technology transfer, i.e., there is more chance that an ISE design will actually be deployed and used, and therefore has some impact.

Concept 4. *A base ISA (resp. base micro-architecture) is deemed* **ISE-aware** *if it explicitly facilitates ISEs. Otherwise, it is deemed* **ISE-oblivious**.

Concept 5. *A tool-chain or component thereof, e.g., a compiler, is deemed* **ISE-aware** *if it can accept (resp. produce) instructions from a given ISE as input (resp. output), versus those in the base ISA alone. Otherwise, it is deemed* **ISE-oblivious**.

Of course, ASIPs take the concept of ISE-awareness to an extreme. However, due to the overlap, we simply treat ASIP-based ISEs as any other ISE: doing so renders specific use of related technologies (e.g., Garp [CHW00], Tensilica Xtensa [Gon00], Synopsys Processor Designer [NST10], etc.) out of scope. Note that being ISE-oblivious means extension of the base ISA is likely more difficult (due to the lack of facilitation), *not* impossible.

Concept 6. *An ISE is termed* **algorithm-specific** *when explicitly designed to support a specific algorithm, or* **algorithm-agnostic** *otherwise.*

Concept 7. *An algorithm-specific ISE is said to offer* **single-algorithm** *(resp.* **multi-algorithm***) support if the scope of specificity is $n = 1$ (resp. a set of $n > 1$) algorithm(s).*

ISEs are, by definition, intended to support domain-specific workloads. For cryptographic workloads, however, specialisation *within* the domain, i.e., on a per-algorithm basis, can be reasonable. For example, consider an ISE supporting use of an S-box in block ciphers: it might support any block cipher which uses an S-box (algorithm-agnostic), a specific family of block ciphers (algorithm-specific, multi-algorithm), or a specific block cipher (algorithm-specific, single-algorithm). In part, adopting a per-algorithm approach can be justified by the role that standardisation play in cryptography. This means a relatively small set of (standard) algorithms have strong practical relevance (e.g., NIST FIPS-197 [NIST01] and Federal Information Processing Standard (FIPS)-180 [NIST15] for AES and SHA256, and SP 800-38d [NIST07] for the GCM mode of operation). Selection between them is further guided by their inclusion in standard parameter sets (e.g., the Transport Layer Security (TLS) [Res18, Section B.4] cipher suite `TLS_AES_128_GCM_SHA256`). Note that multi-algorithm support may occur implicitly, if, for example, an algorithm takes advantage of an existing ISE by using a platform-aware design approach. For example, AES-NI can be used beyond AES itself: Bos, Özen, and Stam [BOS11] explore application within hash function constructions, Saarinen [Saa] uses it to implement of the SM4 block cipher, the Grøstl hash function [GKM+11] uses the S-box, and the YAES [BV14] authenticated

encryption scheme uses a full round. Therefore, the definitions relate more to the explicit, by-design support offered by a given ISE.

Given an algorithm, one can attempt to classify the granularity of support by an associated ISE. Doing so is common in the specific case of an (iterative) block cipher algorithm, where one might use a definition such as the following

Concept 8. *Specific instances of support can be classified as* **all-rounds** *(or* **full-cipher***, e.g., for an entire encryption operation),* **one-round** *(or* **full-round***, e.g., for one round within an encryption operation), or* **part-round** *(e.g., for one step within a round).*

For example, the AVR base ISA [Inc20, Table 4-2] includes an all-rounds instruction to support encryption or decryption using Data Encryption Standard (DES); extensions of the RISC-V base ISA includes part-round and one-round instructions to support encryption or decryption using AES, in Zk [RIS24, Section 31] and Zkv [RIS24, Section 32] respectivly. Analagous definitions are clearly possible for other classes of algorithm, but will differ re. terminology, e.g., with respect to the term used for an iterative step.

Concept 9. *An ISE is termed* **discoverable** *if the base ISA includes a mechanism by which (non-)support for it can be programmatically determined.*

Concept 10. *In the same way as instructions in a base ISA, those in an ISE are termed* **stateful** *if they depend on some form of state; otherwise, they are termed* **non-stateful** *(or* **stateless***).*

Note that the state here refers to something, e.g., control or status data, *other than* the input operands, and that stateful ISEs span cases where 1) new state is *added* by the ISE versus 2) existing state is defined by the ISA and simply *used* by the ISE.

Choquin and Piry [CP20, Page 2] use a definitional framework to classify accelerator technologies. We use it as a starting point to differentiate ISEs from related technologies, and to classify their implementation. First, the ISE implementation must be integrated with a base micro-architecture; this renders decoupled, memory mapped peripherals and co-processors (e.g., CryptoBooster [MTR+99], Celator [FPP08], CCproc [TSP09], FastCrypto [SA10], and Falcon [KLA+19], HACC-V [Sou22]) out of scope. Second, where such integration is evident, there are (at least) two sub-classes which are further illustrated by Figure 1:

Concept 11. *A* **tightly-integrated ISE** *implementation exists "within" the base micro-architecture, and so shares characteristics with the base ISA and micro-architecture. Indicative characteristics include direct access to state defined by the base ISA; bandwidth and throughput limited by, e.g., ports on general-purpose register file; instructions executed internally by the base micro-architecture, e.g., interleaved with instructions from the base ISA; instruction execution has lower-latency, e.g., < 3 cycles.*

Concept 12. *A* **loosely-integrated ISE** *implementation exists "alongside" the base micro-architecture, with interaction between them occurring, e.g., via an integration interface. Indicative characteristics include indirect access to state defined by the base ISA; bandwidth and throughput limited by the integration interface; instructions executed externally to the base micro-architecture; instruction execution has higher-latency, e.g., ≥ 3 cycles.*

Concept 13. *The ISE implementation is usually captured in a set of hardware components then integrated into a base micro-architecture. The term used to describe said components varies, but* **Application-specific Functional Unit (AFU)** *is a common example.*

Discussion: ISE versus non-ISE? Beyond classification based on the above, precisely differentiating an ISE from a non-ISE can be difficult and perhaps even counterproductive. One approach would be to demand ISE-based instructions are "similar to" ISA-based instructions, in the sense the former have no special-case features, e.g., in terms of

implementation constraints, execution semantics, etc., relative to the latter. However, consider two examples:

- Steinegger and Primas [SP21] present an ISE for Ascon, based on RISC-V; [SP21, Figure 1] details implementation of a dedicated AFU for Ascon-p, i.e., the Ascon permutation, which is tightly-integrated with the base micro-architecture. The ISE itself includes one instruction, which essentially supports computation of an entire Ascon round via the AFU. As a result, the AFU assumes the state is read from (resp. written to) a set of 10 general-purpose registers; access to those registers is hard-wired, and must avoid pipeline hazards (which are not resolved by existing forwarding logic).
- Kumar and Paar [KP04] present an ISE for arithmetic in $\mathbb{F}_{2^{163}}$, based on AVR; [KP04, Figures 1+2] detail implementation of a dedicated AFU for said arithmetic, which is tightly-integrated with the base micro-architecture. For example, although the AFU *appears* tightly-integrated with other components, it 1) operates asynchronously: once an operation is initiated, polling is used to test for completion (i.e., during which time other instructions can be executed, although "*the software has to take special care not to call the custom hardware until the multi-cycle operation is completed*"), and 2) has a dedicated memory interface used to load and store 163-bit input and output words.

On one hand, the instructions specified in both designs clearly have special-case features that are (arguably) more aligned with those of a co-processor than they are an ISE. On the other hand, however, provided those features are acceptable, both designs are still viable and may be effective whether deemed an ISE or not: [SP21, Table 1 + Section 4] highlight that a factor of 1.1 area overhead affords a very significant, factor of 50 improvement in execution latency for Ascon, for example.

We therefore argue it is more productive to define the term ISE as broadly as possible, e.g., as "*any mechanism within the (base or extended) micro-architecture, which is accessible via synchronous execution of an instruction not included in the base ISA*". Note that synchronous should be understood in relation to the surrounding instruction stream, and that such a definition excludes use of a loosely-integrated accelerator via instructions which simply manipulate either external (i.e., memory-mapped) or internal control registers.

Discussion: RISC versus CISC? It seems reasonable to say that although the terms Reduced Instruction Set Computer (RISC) and Complex Instruction Set Computer (CISC) can be understood intuitively, there is room for interpretation with respect to their definition. That is, when classifying a design as either RISC or CISC, it is often useful to consider a set of indicative characteristics and/or an underlying technical principles. rather than a terse, rigid definition. For example, Patterson and Séquin [PS81, PS82] offer a set of characteristics stemming from the seminal Berkeley RISC project: these can be summarised as 1) single-cycle instruction execution, 2) uniformity with respect to instruction encoding, 3) limited addressing modes (e.g., memory access only via dedicated load and store instructions), and 4) support for high-level programming languages. Likewise, Waterman [Wat16, Chapter 3] states that the principle guiding RISC-V "*was to make an ISA suitable for nearly any computing device*".

On one hand, a base ISA plus ISE is (still) an extended ISA, so can clearly be classified using the same approach. On the other hand, however, it is crucial to avoid interpreting "extended" as "more complex" and thus CISC-like. That is, although cryptographic ISE designs exist which *are* complex, complexity is not *implied* by domain-specificity: designs *also* exist which align with the ISA, e.g., by retaining RISC-like characteristics.

Discussion: does generality *really* matter? At face value, an ISE being less algorithm-specific seems an obvious advantage. Alongside any subjective argument around generality implying elegance, it almost certainly supports wider applicability and therefore higher

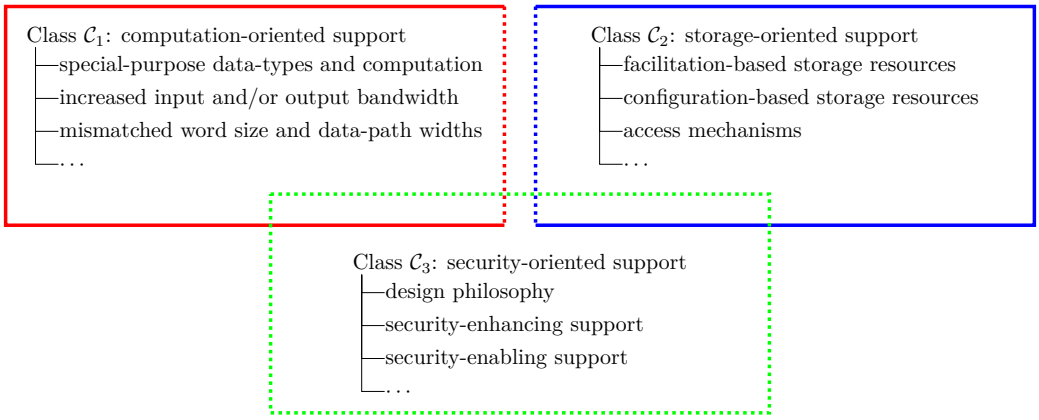


Figure 2: A diagrammatic summary of the 2-level taxonomy presented in Section 3.

utilisation. However, commercially deployed cryptographic ISEs *are* often specific to a given algorithm. ISE-based support for AES summarised in Section 1 is an example of this fact: AES is (presumably) deemed an important, valuable enough workload that any disadvantages of algorithm specificity are moot. As a result, we argue that generality should be viewed more as a *merit* than used as a *metric*: any lack of generality should not be viewed as disadvantageous, *iff.* some algorithm-specific ISE delivers sufficient value.

3 A comprehensive taxonomy of cryptographic ISEs

In this Section, we present a 2-level taxonomy of existing work on cryptographic ISEs summarised by Figure 2: leveraging the framework developed in Section 2, we organise said work into

1. a set of major classes for ISEs that offer computation-, storage-, and security-oriented support, then,
2. a set of minor classes for ISEs that address the same underlying challenge and/or adopt a similar approach.

Within each class we adopt a uniform structure by first providing an explanation of the underlying concept, and then, second, providing a short précis of ISE designs which realise said concept: note that we defer more detailed coverage to the full version of this paper [CGM+24].

As well as being significantly more contemporaneous than existing alternatives, we argue our taxonomy increases the level of breadth (e.g., coverage of challenges and approaches) *and* depth (e.g., level of detail). For example, Nahum et al. [NOOS95, Section 4] highlight "*three basic problems*" which they proposed to address via an ISE, namely "*operations on sub-wordsize units, operations on super-wordsize units, and operations of groups other than that of integers*"; this explicitly computational focus omits modern ISEs which target security-oriented support. Likewise, Regazzoni and Ienne [RI16] highlight ISEs focused on providing increased performance (see [RI16, Section III]) and resilience against side-channel attack (see [RI16, Section IV]); this implicitly computational focus omits modern ISEs which target storage-oriented support. That said, we acknowledge that our taxonomy (as any) naturally has advantages *and* disadvantages. For example, on one hand it offers a best-effort attempt to group similar ISEs together, but, on the other hand, it means a given ISE could plausibly span several classes even where highlighted in one. Figure 2 attempts to hint at this fact: class $\mathcal{C}_3$ overtly overlaps with classes $\mathcal{C}_1$ and $\mathcal{C}_2$ (e.g., a mechanism for enhanced security may be based on alteration of how computation is performed), whereas there is an implicit "gray area" between classes $\mathcal{C}_1$ and $\mathcal{C}_2$ (e.g., a mechanism for addressing

can be viewed computation of an address, but is strongly associated with the storage medium said address is used to access). For class $\mathcal{C}_1$ only, Figure 1 provides a comparative summary of constituent ISEs; we avoid doing so for other classes due to their greater diversity, meaning such comparison is less meaningful.

3.1 Class $\mathcal{C}_1$: computation-oriented support

This major class of ISE relates to computation-oriented support. It contrasts, e.g., with storage-oriented support, in the sense it broadly relates to active or "in-flight" data, i.e., input data and the computation on it which yields output data. From the perspective of the base core, implementation of such an ISE will typically focus on *internal* components, e.g., enhanced forms of Arithmetic and Logic Unit (ALU) which realise the computation required.

3.1.1 Class $\mathcal{C}_{1.1}$: special-purpose data-types and computation

Section 1 cites a focus on general-purpose workloads in existing base ISAs, which typically means support for integer (and perhaps floating-point) data-types plus arithmetic and logic (or ALU-like) operations on them. In contrast, cryptographic workloads make often use of data-types and representations (e.g., stemming from richer Mathematical structures) which lack native support in the base ISA for associated operations. This fact provides perhaps the most compelling motivation, which has led to a broad class of ISEs that support special-purpose alternatives focused on cryptography.

Examples. Various work has proposed ISEs to support candidates in either the NIST SHA-3 process [CBG12] (e.g., BLAKE, Grøstl, JH, Keccak, and Skein), or the NIST Light-Weight Cryptography (LWC) process [CGM+22, GL24] (e.g., Ascon, Elephant, GIFT-COFB, Grain-128AEADv2, PHOTON-Beetle, Romulus, Sparkle, TinyJAMBU, and Xoodyak). Alkim et al. [AEL+20] present an ISE for RISC-V, which supports computation in $\mathbb{F}_q$ for small, prime q; examples include; 12-bit $q = 3329$ and 14-bit $q = 12289$ motivated by use in Kyber and NewHope; Kiaei et al. [KMD+20] (then later Kiaei, Conroy, and Schaumont [KCS23]) present SKIVA, an ISE and associated implementation designed to support implementations based on the use of (aggregated) bit-slicing. Further examples include [Saa19, BEM+15, EMOC20, GM23, GP12, VSI+19, EAD+22, DBB+23, MPP21, MNP+21, Saa20, SP21, AO21, BSMH25, BMA00, MD07, FL04, PSP08, TG04, GS04, GK03b, GK03a, VPG07, NOOS95, KGM21, KGRM23, GKP04, UK24, EKP+13, Elb07, TGSD20, GGP08, ITAO20, FSS20b, LTQ+24, OFP+24, LQYW24, FSS20a, MBB+23, GLM24, NDZ+21, WCLW24, DPM+25, KH25].

3.1.2 Class $\mathcal{C}_{1.2}$: increased input and/or output bandwidth

Consider a base ISA with an n-address instruction format: it allows instructions to specify up to n general-purpose register addresses, such that $n = n_s + n_d$ for n_s sources and n_d destinations (or targets). Although the case where $n = 2 + 1 = 3$ is common, support for n-address instruction formats more generally, e.g., cases where $n > 3$, involves a non-trivial trade-off between a variety of factors. For example, increasing n can increase data-flow bandwidth which, in turn, facilitates *richer* forms of computation as a result of access to more data. Crucially, however, increasing n can also 1) increase encoding pressure (in the sense that more bits are required to encode the associated register addresses, and hence instruction), and 2) increase data-flow complexity (in the sense that either n register file ports or multi-cycle execution must be supported). In order to realise the clear advantage therefore, any challenges related to the associated *dis*advantages must be addressed.

Table 1: Overview of Class $\mathcal{C}_1$ examples.

Design[1]	ISA	Support[2]	Target	Stateful	HW[3]	SW[4]
[MNP+21]	RISC-V	○	AES	✗	●	⊙
[GM23]	RISC-V	○	AES	✗	◉	◉
[ISP24]	RISC-V	○	AES	✔	◉	●
[EMOC20]	Xtensa	○	AES, DES, SHA-256	✗	●	◉
[BEM+15]	SPARC V8	○	AES, SHA-1	✔	◉	-
[Saa20]	RISC-V	○	AES, SM4	✗	●	◉
[AO21]	RISC-V	○	Ascon	✗	-	○
[SP21]	RISC-V	○	Ascon	✗	◉	●
[MPP21]	RISC-V	○	ChaCha	✗	●	⊙
[BSMH25]	RISC-V	○	Keccak	✔	◉	◉
[GP12]	RISC	○	PRESENT, SEA, XTEA	✗	-	⊙
[VSI+19]	VSCPU	○	PRESENT	✗	-	●
[EAD+22]	RISC-V	○	PRESENT, PRINCE	✔	●	◉
[DBB+23]	RISC-V	○	QARMA	✗	●	●
[Saa19]	RISC-V	○	SNEIK	✗	◉	◉
[CBG12]	PIC24	○	NIST SHA-3 finalists	✗	●	⊙
[CGM+22]	RISC-V	○	NIST LWC finalists	✗	◉	⊙
[GL24]	Xtensa	○	NIST LWC finalists	✔	◉	◉
[BMA00]	Alpha	●	bit-manipulation	✔	-	○
[MD07]	TinyRISC	●	bit-manipulation	✗	-	-
[NOOS95]	RISC	●	$\mathbb{F}_2[\mathbf{x}]$	✗	-	○
[FL04]	PLX	●	$\mathbb{F}_2[\mathbf{x}]$	✗	-	◉
[GK03a]	RISC	●	$\mathbb{F}_2[\mathbf{x}]$	✗	-	-
[TG04]	SPARC V8	●	$\mathbb{F}_2[\mathbf{x}]$	✗	-	○
[PSP08]	N-Core	●	$\mathbb{F}_2[\mathbf{x}]$	✗	◉	⊙
[GK03b]	RISC	●	$\mathbb{F}_2[\mathbf{x}]$, $\mathbb{F}_p$	✗	-	○
[GS04]	MIPS32	●	$\mathbb{F}_2[\mathbf{x}]$, $\mathbb{F}_p$	✗	-	◉
[VPG07]	SPARC V8	●	$\mathbb{F}_2[\mathbf{x}]$, $\mathbb{F}_3[\mathbf{x}]$, $\mathbb{F}_p$	✗	-	●
[GKP04]	MIPS32	●	$\mathbb{F}_{p^k}$ (prime $p = 2^n - c$)	✗	-	○
[AEL+20]	RISC-V	●	$\mathbb{F}_q$ (small prime q)	✔	●	○
[KGM21]	RISC-V	●	$\mathbb{F}_{2^m}$ (small m)	✔	●	◉
[KGRM23]	RISC-V	●	$\mathbb{F}_{2^m}$ (small m)	✔	●	◉
[Elb07]	SPARC V8	●	binary matrix	✗	-	◉
[EKP+13]	AVR	●	binary matrix	✔	-	○
[UK24]	RISC-V	●	binary matrix	✔	○	⊙
[TGSD20]	RISC-V	●	nibble-wise matrix	✔	○	●
[GGP08]	CRISP	●	bit-slicing	✔	●	○
[KMD+20]	SPARC V8	●	bit-slicing	✗	●	-
[KCS23]	RISC-V	●	bit-slicing	✗	●	⊙
[ITAO20]	RISC-V	○	NTRU	✔	⊙	○
[FSS20b]	RISC-V	◉	Kyber, NewHope, Saber	✗	⊙	◉
[FSS20a]	RISC-V	○	LAC	✔	○	◉
[NDZ+21]	RISC-V	○	Kyber, Dilithium	✔	●	⊙
[MBB+23]	RISC-V	○	Kyber, Dilithium	✗	●	○
[DPM+25]	RISC-V	◉	ML-KEM, ML-DSA, SLH-DSA, HQC	✗	◉	⊙
[LQYW24]	RISC-V	○	Kyber	✗	●	○
[GLM24]	RISC-V	○	Kyber	✗	◉	○
[LTQ+24]	RISC-V	○	Dilithium	✔	◉	◉
[WCLW24]	RISC-V	○	Falcon	✔	○	○
[KH25]	RISC-V	○	Classic McEliece	✔	-	⊙
[OFP+24]	RISC-V	◉	X25519, Saber	✗	⊙	◉
[SL00]	PA-RISC	●	bit permutation	✗	-	⊙
[Gro03]	MIPS32	●	multi-precision arithmetic	✗	-	○
[CFG+24]	RISC-V	●	multi-precision arithmetic	✗	●	○
[US24]	OBTN	●	$\mathbb{F}_q$ (small prime q, SIMD)	✗	-	○
[AOP+25]	OBTN	●	$\mathbb{F}_q$ (small prime q, SIMD)	✔	◉	⊙
[LTZ+25]	RISC-V	●	multi-precision arithmetic (SIMD)	✔	-	○
[RS16]	ARM NEON	○	Keccak (SIMD)	✗	●	⊙
[LMP23]	RISC-V	○	Keccak (SIMD)	✗	○	-
[LMP22]	RISC-V	○	Kyber (SIMD)	✔	○	●
[YSZ+24]	RISC-V	○	Kyber, Dilithium (SIMD)	✔	○	◉
[YLW+25]	RISC-V	○	SLH-DSA (SIMD)	✔	○	◉

[1] All examples are tightly-integrated.
[2] Support: ● algorithm-agnostic; ◉ multi-algorithm; ○ single-algorithm.
[3] Hardware overhead (vs. base core): ● very low; ◉ low; ◉ moderate; ⊙ high; ○ very high.
[4] Software improvement (vs. ISA-only baseline): ● very high; ◉ high; ◉ moderate; ⊙ low; ○ very low.

Lee, Yang, and Shi [LYS04] observe that cryptographic workloads can be well positioned to take advantage of this potential, because they are naturally specified in terms of Multi-word Operands, Multi-word Results (MOMR) operations: when those operations are mapped more directly onto instructions, their execution will typically be more efficient.

Examples. Lee and Choi [LC08] propose the Register File Extension for Multi-word and Long-word Operation (RFEMLO) where a group of $n = 2^l$ contiguous register addresses are derived from a single register address i plus a level l, i.e., $(i, l) \mapsto \langle i, i+1, i+2, \ldots, i+2^l-1 \rangle$. Fiskiran and Lee [FL04] present an ISE is set within the context of the PLX micro-architecture; their case-2 variant defines two instructions, namely `bfmul.lo` and `bfmul.hi`, which respectively write the LSBs and MSBs of a computed product into a general-purpose register. Further examples include [GGM+21, SP21].

3.1.3 Class $\mathcal{C}_{1.3}$: mismatched word size and data-path widths

Given a base ISA which specifies a w-bit word size, the first two problems identified by Nahum et al. [NOOS95, Section 4] relate to the use of operations whose natural word size is either less-than (i.e., "*sub-wordsize*") or greater-than (i.e., "*super-wordsize*") w. Put another way, such operations would be more naturally processed by a data-path, and so natural word size of some $w' \neq w$ bits. For example, the state (or block size) of a block cipher design might be 64-bit (e.g., PRESENT [BKL+07]) or 128-bit (e.g., AES [NIST01]) and so $w' > w = 32$; the computation performed on that state might be 1-bit (e.g., DES [NIST99]), 4-bit (e.g., PRESENT [BKL+07]), 8-bit (e.g., AES [NIST01]), or 16-bit (e.g., IDEA [LM90]) oriented and so $w' < w = 32$. In the case of AES, for example, the choice is explicitly rationalised: although still a compromise, specifying 8-bit oriented computation supports versatility [DR02, Section 5.1.4], i.e., "*the ability to be implemented efficiently on different platforms*". In other cases (e.g., Speck [BSS+13] and Simon [BSS+13], RC5 [Riv94] and RC6 [RRSY98]), this approach is taken further by allowing specification of the natural word size as a parameter.

In the $w' > w$ case, it is common to represent a w'-bit operand using a sequence of w-bit words; associated challenges include efficiently dealing with any interaction, e.g., carries, between those words. In the $w' < w$ case, it is common to harness existing support for SIMD or SWAR (i.e., packed) operations. However, non-orthogonality in that support is a common challenge: in early work of this type Acar [Aca97, Section 5.4.1] noted, for example, that MMX "*lacks certain instructions such as* 16*-bit and* 32*-bit unsigned multiply and multiply-add*" which made harnessing it in cryptographic use-cases more difficult. Arguably this issue has improved over time, but support for $w' < 8$ is still uncommon.

Examples. Various work has proposed ISEs to support bit permutation i.e., permutation of w'-bit sub-words for $w' = 1$; early instances include that of Shi and Lee [SL00]. Various work has proposed ISEs to support multi-precision integer arithmetic, i.e., arithmetic with w'-bit integers for $w' > w$; early instances include that of Großschädl [Gro03]. Abdulrahman et al. [AOP+25] propose an ISE to support Kyber and Dilithium as implemented for the OpenTitan Big Number (OTBN); their ISE allows SIMD-like computation across w'-bit sub-words of the w-bit data registers, where $w = 256$, and, e.g., $w' \in \{16, 32\}$, meaning $w' < w$. Further examples include [BMA00, TGSD20, CGM+22, GS04, CFG+24, YSZ+24, LMP23, RS16, LMP22, YLW+25, US24, LTZ+25].

3.2 Class $\mathcal{C}_2$: storage-oriented support

This major class of ISE relates to storage-oriented support. It contrasts, e.g., with computation-oriented support, in the sense it relates to inactive or "at-rest" data, i.e., data stored in some resource. From the perspective of the base core, implementation of

such an ISE will typically span *internal* and *external* components, e.g., additional registers or alteration of the wider memory hierarchy.

3.2.1 Class $\mathcal{C}_{2.1}$: facilitation-based storage resources

This minor class of ISE *facilitates* instruction execution using some storage resource. For example, the MIPS32 `mul` [MIPS01b, Page 169] instruction semantics are $\mathsf{GPR}[\texttt{rd}] \leftarrow \mathrm{LSB}_{32}(\mathsf{GPR}[\texttt{rs}] \times \mathsf{GPR}[\texttt{rt}])$. In contrast, the `mult` [MIPS01b, Page 169] instruction has the following semantics $(\mathsf{GPR}[\texttt{lo}], \mathsf{GPR}[\texttt{hi}]) \leftarrow \mathsf{GPR}[\texttt{rs}] \times \mathsf{GPR}[\texttt{rt}]$. Put simply, the former explicitly uses *one* 32-bit general-purpose register to capture the least-significant 32 bits of the full product; the latter implicitly uses *two* 32-bit special-purpose registers `hi` and `lo` to capture the full product (`hi` captures the more- and `lo` the less-significant 32-bits respectively). As such, the latter is facilitated by addition of these registers plus a suite of associated instructions, i.e. `mfhi` [MIPS01b, Page 146], `mflo` [MIPS01b, Page 147], `mthi` [MIPS01b, Page 166], and `mtlo` [MIPS01b, Page 167], which support access.

Examples. Großschädl [Gro03] considers implementation of multi-precision integer arithmetic on MIPS32, using the (existing) special-purpose registers `hi` and `lo`; to support more efficient accumulation of, e.g., partial products, `hi` is extended from 32 to 33 bits. Fiskiran and Lee [FL04, Section 3.3] consider implementation of arithmetic in $\mathbb{F}_{2^8}$, exploring variants of the `bfmul` instruction for carry-less multiplication. The variant termed case 1 introduces special-purpose registers `RH` and `RL`; to capture the more- and less-significant words of the product.

3.2.2 Class $\mathcal{C}_{2.2}$: configuration-based storage resources

This minor class of ISE *configures* instruction execution using some storage resource. That is, one can distinguish between 1) dynamic state that is operated on by the instruction, or 2) static state that configures (or controls) the instruction, but is not operated on per se. The latter case could be further characterised as being either semi-static, e.g., held within a register, memory, or some special-purpose, or fully-static, e.g., encoded as an immediate within the instruction. Within the context of cryptographic ISEs, a common use-case for such state is to provide a compromise between general-purpose and special-purpose behaviour: the idea is to use the state as an additional input which parameterises an otherwise special-purpose operation, thus rendering it more broadly applicable.

Examples. Tehrani et al. propose an ISE that supports parallel application of a 4-bit S-box [TGS+19, Section III.A], i.e., a mapping $\{0,1\}^4 \rightarrow \{0,1\}^4$, to each nibble in a 32-bit word, e.g., for block cipher implementation; semi-static configuration for the S-box is held in "*three CSR registers*". Likewise, they propose an ISE that supports nibble-wise matrix-vector multiplication [TGS+19, Section III.D], the (compressed) matrix operand is held in "*eight 32-bit CSR registers*". Alkim et al. [AEL+20] propose an ISE that supports arithmetic in $\mathbb{F}_q$, and explore variants where q and associated parameters are either 1) static, or 2) dynamic: the latter case is supported by state stored in "*internal registers*" and accessed by special-purpose instructions (namely `ffset` and `ffget`). Further examples include [GGP08, KGM21].

3.2.3 Class $\mathcal{C}_{2.3}$: access mechanisms

This minor class of ISE emphasises a mechanism for access *to* some storage resource, rather than addition of the storage resource itself; there can be an overlap between such remits, however, and therefore between this and, e.g., classes $\mathcal{C}_{2.1}$ and $\mathcal{C}_{2.2}$.

Examples. Fiskiran and Lee [FL01] assess the impact of addressing mode on the efficiency of cryptographic implementations, describe an ISE-like "*[load] instruction which combines index extraction, index scaling and [the] memory access*". Fiskiran and Lee [FL05b, FL05a] (then later Lee and Chen [LC10]) describe an ISE for efficient look-up table access, based on a so-called Parallel Table Lookup (PTLU) module that houses 8 on-chip, 256-entry look-up tables with 32-bit entries. Further examples include [BMA00, AO21, HYL08, GGH+11, HGTW05].

3.3 Class $\mathcal{C}_3$: security-oriented support

Per Section 1, a distinguishing characteristic of cryptographic workloads is their need to consider security as a quality metric. In selected cases, this fact is reflected by security-oriented analysis of a given ISA (see, e.g., that of Lu [Lu21] for RISC-V) and wider methodology (see, e.g., that of Belaïd et al. [BCM+23]). Consideration within the context of ISE design is likewise important. From a positive perspective, for example, note that the implementation of an ISE is necessarily set within the context of some micro-architecture. Use of the ISE can therefore allow 1) access to or 2) control over resources that would be impossible using the ISA, and can be leveraged to provide functionality or enforce behaviour which is security-enhancing in some way. From a negative perspective, however, note that Saab, Rohatgi, and Hampel [SRH16] concluded that naive use of AES-NI allows a specific form of attack. One can argue reasonably that a similar attack applies to an implementation that uses the ISA alone, so one cannot "blame" the ISE. However, there is a crucial difference, in the sense that various countermeasures are viable in an ISA-based implementation but non-viable in an ISE-based implementation: this fact stems from flexibility of software versus hardware (i.e., micro-architectural implementation of the ISE), leading to the challenge of designing ISEs which can be composed with countermeasures.

3.3.1 Class $\mathcal{C}_{3.1}$: design philosophy

Some work can be classified as taking a broader, and higher-level approach, in the sense it addresses challenges in (cyber-)security more generally (which then impact cryptography in specific instances) by fundamentally re-evaluating what an ISA should be or do within that context. Most examples are arguably better framed as altering versus extending the base ISA, although intersect with reasonable definitions of ISE even so.

Examples. Ge, Yarom, and Heiser [GYH18] present the case for a "*new security-oriented hardware/software contract*" they dub the augmented ISA (aISA), a central feature of which is (selected) exposed of micro-architectural detail to allow transparency and control over instruction execution. Escouteloup et al. [EFLL20] aim to "*prevent timing side-channels, strengthen control flow integrity and ensure micro-architectural state isolation*" by proposing RV32S, a "secure" ISA design based on RISC-V. Further examples include [SFSG23, ZSM19, YHHF19].

3.3.2 Class $\mathcal{C}_{3.2}$: security-enhancing support

This minor class of ISE emphasises security-*enhancing* support; the level of security of an implementation using it is *directly* enhanced by the ISE itself. An example would be ISEs which realise or support realisation of some form of implementation attack countermeasure.

Examples. Bayrak et al. [BVIB12] describe an ISE which supports (constrained) random reordering (or "shuffling") of instruction execution, and so constitutes a hiding-based countermeasure against certain side-channel attacks. Gao et al. [GGM+21] (then later Krausz et al. [KLS+23]) describe an ISE for RISC-V which includes a suite of instructions

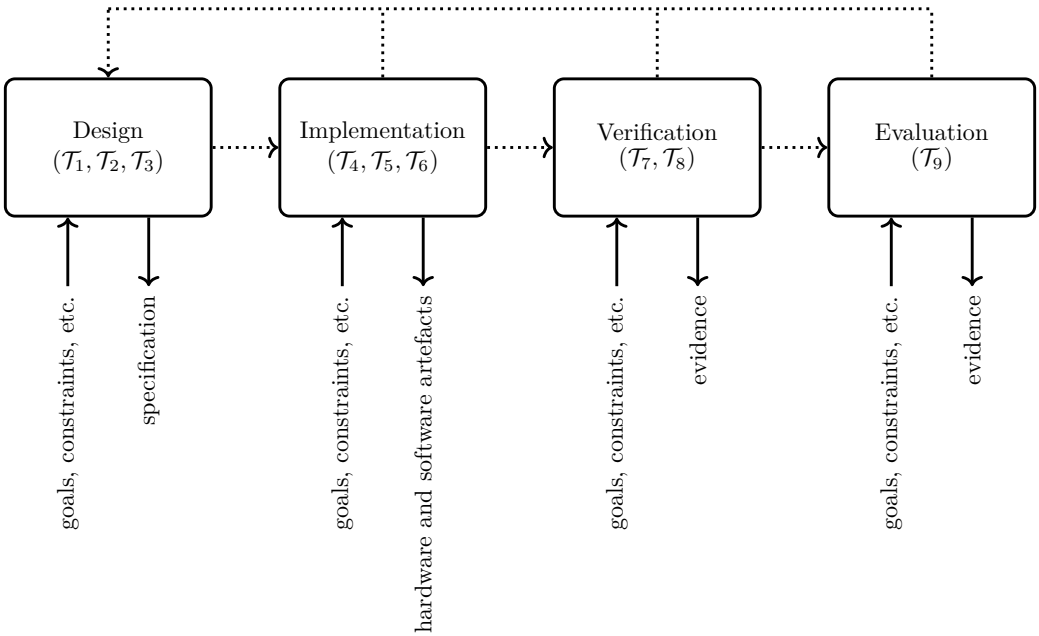


Figure 3: A diagrammatic summary of the development process presented in Section 4. Note that 1) the iterative, waterfall-like workflow is described using dashed arrows, and 2) each phase is annotated with indicative inputs and outputs (or outcomes).

whose semantics mirror so-called "gadgets" used in side-channel countermeasures based on Boolean masking. Wistoff et al. [WSG+20] describe an ISE for RISC-V named `fence.t`, which "*that flushes micro-architectural state*" with the aim of preventing certain micro-architectural covert- and side-channel attacks. Further examples include [ZQL+24, FBR+21, MP21, LT23, KS20, KLS+23, CB23, TCG+25, CLB+25, LHP20, GMPP20, CPW24, TG07a, TKS10, VCS22, KMD+20, RCS+09, THM07, REP+09].

3.3.3 Class $\mathcal{C}_{3.3}$: security-enabling support

This minor class of ISE emphasises security-*enabling* support: the level of security of an implementation using it is *indirectly* enabled by the ISE. An example would be ISEs which supports (pseudo-)randomness generation, harnessed by said implementation to (more efficiently) generate (higher quality) key material.

Examples. Chen et al. [CZQ+24] describe an ISE for RISC-V named RISecure-PUF, which constitutes an interface to some underlying Physical Unclonable Function (PUF) implementation. Schwarz et al. [STRG24] describe an ISE for RISC-V named KeyVisor, which allows software to manage hardware-resident key material (thereby preventing misuse, such as exposure via the memory hierarchy) and invoke hardware-supported cryptographic operations using it. Saarinen, Newell, and Marshall [SNM22] describe an ISE for RISC-V (later ratified as part of the standard Zk extension [RIS24, Section 31]), which constitutes an interface to some underlying entropy source implementation. Further examples include [XLY+22, OGM+13, Int18, LCW18, AMD17, Cam80, Pow15, HKM12].

4 A structured cryptographic ISE development process

In this Section, we present a structured cryptographic ISE development process which captures both generic principles and specific techniques and technologies; we pitch the

process as "idealised" in the sense that elements of it may or may not be applicable in a given context, but with the overarching goal of guiding and so maximising the quality of future work. The structure and content of said process is based on concrete, plus both positive and negative evidence stemming from our analysis of instances in Section 3. As motivation, consider the following limited set of examples (which we add to throughout the Section). First, instances such as [KGM21] introduce state plus instructions to write it, but *not* to read it and so without considering, e.g., how it might be context-switched; Section 4.1.2 collects a range of similar design requirements, and explores the potential impact of their (non-)consideration. Second, instances such as [BLCN20] lack a clear specification of instruction semantics; Section 4.1.3 highlights how to, and the value of doing so. Third, instances such as [AO21] lack a hardware implementation and thus any evaluation with respect to, e.g., area; Section 4.4 establishes an extensive set of metrics by which a given ISE might be evaluated, which act to illustrate the complexity of trade-offs (cf. [Gue09, Section 4]) on which effective design decisions can depend.

At a low level, the ISE development process can be described as constituting, and typically iteration over a set of tasks:

- **Task** $\mathcal{T}_1$: Select a base ISA.
- **Task** $\mathcal{T}_2$: Identify opportunities for extending the ISA using an ISE.
- **Task** $\mathcal{T}_3$: Create an ISE specification, including components for 1) state, 2) semantics of instructions, and 3) encoding of instructions.
- **Task** $\mathcal{T}_4$: Implement a suitable set of AFUs.
- **Task** $\mathcal{T}_5$: Implement alterations to base micro-architecture to permit execution of instructions specified in the ISE.
- **Task** $\mathcal{T}_6$: Implement a mechanism which allows development of software that uses instructions specified in the ISE.
- **Task** $\mathcal{T}_7$: Verify the hardware implementation is correct.
- **Task** $\mathcal{T}_8$: Verify the software implementation is correct.
- **Task** $\mathcal{T}_9$: Relative to a baseline without it, i.e., using the base ISA alone, evaluate the ISE with respect to pertinent quality metrics.

At a high(er) level, the same tasks can be grouped into four phases, namely

Tasks	$\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3$	$\mapsto$	design	$\in$	Section 4.1
Tasks	$\mathcal{T}_4, \mathcal{T}_5, \mathcal{T}_6$	$\mapsto$	implementation	$\in$	Section 4.2
Tasks	$\mathcal{T}_7, \mathcal{T}_8$	$\mapsto$	verification	$\in$	Section 4.3
Task	$\mathcal{T}_9$	$\mapsto$	evaluation	$\in$	Section 4.4

which we illustrate in Figure 3 and explore in the cited Sections. Note that most of the tasks presented above apply to ISA design as well as ISE design; likewise, most apply to non-cryptographic as well as cryptographic ISE design. To some extent we view this as a natural side-effect of, e.g., an ISE extending an ISA, but, equally, stress that discussion in the following Sections focuses on many points that are specifically relavent to cryptographic ISE design.

4.1 Design

Patterson and Ditzel [PD80] and Chisnall [Chi24] attempt to capture experience of and advice about ISA design. Most points in both focus on general-purpose ISAs, but, in a broad sense, most are also applicable to cryptographic ISE design. However, it seems unlikely that a definitive approach, or codified advice that "solves" the ISE design problem [GB11] does or even can exist: both the problem and any (candidate) solutions are simply too domain- and context-dependent. Rather, one must employ some form of structured methodology

(see, e.g., Puttmann et al. [PSP08, Section 3]) to understand and explore the design space. In this Section we attempt to capture elements of such an approach.

4.1.1 ISA identification

A choice of ISA may or may not be possible, depending on the context. Different constraints will stem from an ISA being ISE-aware or ISE-oblivious however, consideration of which before and/or during the design process is important even when such a choice is impossible. For example, RISC-V [RIS24] is ISE-aware as a by-design feature: one of the stated goals [RIS24, Chapter 1] is provide "*an ISA supporting extensive ISA extensions and specialized variants*". Various ISAs are ISE-oblivious but still extendable, e.g., because some opcodes are reserved: MIPS32 [MIPS01b, Section 3.1] states, for example that "*[i]mplementations may only use those [reserved] fields [...] for implementation dependent use*" (of which an ISE would be one example). Likewise, the Arm Custom Instruction [CP20] technology is available to licensees of higher-end Cortex-M cores, e.g., Cortex-M33, based on the Armv8-M ISAs. The idea is to "*redefine[s] the coprocessor [...] encoding space to enable custom instructions*" then executed by "*a customizable module inside the processor*".

4.1.2 ISE identification

As already hinted in Section 1, ISA and so ISE design can be a creative task, suggesting that "over-systemisation" can give a false impression of the range and type of viable approaches. That said, a given approach can be usefully characterised in various ways:

Concept 14. *An ISE design approach might be described as either* **algorithm-focused** *or* **implementation-focused***: the latter might focus on requirements of the (specific) algorithm being implemented, whereas the former might focus on a (generic) instruction sequence stemming from implementation of said algorithm.*

Concept 15. *An ISE design approach might be described as either* **exploratory** *or* **analytical***: the latter might derive conclusions via analysis of a fixed set of existing implementations, whereas the former might do so via development and so exploration of bespoke implementations.*

A wide range of work has applied related approaches. Specifically, for example, Stoffelen [Sto19] presents what could be viewed as an instance of the exploratory approach: they implement a range of constructions using the RV32I base ISA, claiming, reasonably, that the resulting evaluation might "*serve to aid design choices for future RISC-V extensions [i.e., ISEs] and implementations*". Tehrani et al. [TGS+19] present what could be viewed as a algorithm-focused, analytical approach: they study a set of 14 light-weight block cipher designs (see [TGS+19, Table 1]), classifying them with respect to the type of components used to realise confusion and diffusion. Chang et al. [CLLG11] present what could be viewed as a implementation-focused, analytical approach: they study a set of 9 designs (see [CLLG11, Section 2.1]), profiling the instruction mix in associated implementations. More generally, many examples employ workload characterisation: Law, Doumen, and Hartel [LDH06], Cazorla et al. [CMM13, CGMM15] and Dinu et al. [DLK+19] focus on block ciphers; Fournel, Minier, and Ubéda [FMU07] focus on stream ciphers; Fiskiran and Lee [FL02], and Branovic, Giorgi, and Martinelli [BGM03] focus on ECC.

Concept 16. *An ISE design approach might be described as either* **manual** *or* **automatic***: the former means the approach is executed by the developer, whereas the latter captures a spectrum of alternatives where some tool(s) offer varying degrees of support.*

A wide range of work has studied related approaches, although few if any cater for cryptography specifically: see, e.g., [PVI02, API03, BCA+04, YM04, SRRJ04, BBD+05, PI05, PAI06, BDPI07, VBI08, ZKB+09, KBIC09, PBI+10, KBCI14, KBB+14]. Clearly a fully-automatic approach would be attractive if it can match (or even approximate) a manual

alternative with respect to pertinent quality metrics. However, even a semi-automatic or tool-assisted design approach might represent an effective compromise between retaining control while addressing challenges such as scale, complexity, etc. For example, Funck, Herdt, and Drechsler [FHD22] consider use of a Virtual Prototype (VP): [FHD22, Figure 1] illustrates what could be viewed as an implementation-focused, analytical approach which is semi-automated by components of the associated framework.

The entire ISE development process is likely iterative, in the sense that successive redesigns might be motivated a posteriori by implementation and evaluation outcomes. However, a priori requirements (or constraints) can often be an effective way to guide the design, and so, e.g., minimise the number of iterations required. In the following, we try to capture some example requirements. We stress that careful assessment of the context is vital, because each example may or may not be relevant and/or useful in said context. For example, one could view many of our examples as related to a scenario where the ISE is intended for adoption, i.e., subsequent integration into an existing ISA. The nature of ISEs means this is clearly not the *only* valid scenario, however: for an alternative where the ISEs will be used in a self-contained product with no dependent stakeholders, some of the same examples could be irrelevant.

Requirement $\mathcal{R}_0$: maximise modularity; enable feature discovery. Although counter-examples (e.g., general-purpose libraries) obviously exist, it is uncommon for a given use-case to require implementations of *all* conceivable cryptographic constructions. With this in mind, a cryptographic ISE which is modular could be viewed as preferable: this property enables specialisation to suit both general-purpose (e.g., include all features) *and* special-purpose (e.g., include some features) use-cases. A positive example would be the RISC-V Zk [RIS24, Chapter 32] extension, which is comprised of various sub-extensions. Support for AES is separated into Zke [RIS24, Chapter 32.3.5] for encryption and Zkd [RIS24, Chapter 32.3.4] for decryption, for example, permitting a decision that omits support for decryption where it is not required. A negative example would be the RISC-V M [RIS24, Chapter 13] extension, which includes support for multiplication *and* division. This prevents the analogous decision, for example omission of support for division where it is not required; the Zmmul [RIS24, Chapter 13.3] extension, a multiplication-only sub-set of M, was subsequently defined to address this. A counter-argument to modularity, however, relates to diversity, i.e., the fact that *multiple* ISA variants can stem from the choices made during specialisation. This raises the issue of compatibility between variants, in the sense that software may be forced to make a worst-case assumption about the features available: if it cannot guarantee a given ISE is available, for example, it must opt for an alternative that is (e.g., by using the base ISA) to offer the associated functionality. Per Section 2, one way to address this issue is via a mechanism for discoverability. For example, support for discoverability would allow software to programmatically determine the features available, then select and use the most effective implementation technique.

If/where the base ISA supports it, therefore, the design of an ISE would ideally take this into account, e.g., by leveraging discoverability to permit identification, selection between, and use of the features it offers. An example of this requirement relates to support by Intel micro-architectures for AES [Gue10] and Secure Hash Algorithm (SHA) [GGY+13], which can be detected using the `cpuid` [Int22a, Chapter 20] mechanism.

Requirement $\mathcal{R}_1$: respect architectural paradigm. A given ISA might be classified using an architectural paradigm; such a classification might be imperfect with respect to specific details, but will often still be useful as a means of articulating general details related to form and/or function. A high-level example would be RISC versus CISC, with lower-level examples including different operand access models, e.g., memory-memory, versus register-memory, versus register-register (aka. load-store).

It seems fair to say that the design of an ISE would ideally not deviate (or at least carefully consider, and limit deviation) from the architectural paradigm of the base ISA. For example, designing an ISE classified as memory-memory for an ISA classified as register-register leads to a number of potential disadvantages such as implementation complexity, and change in programming and/or execution model; one should carefully evaluate these, balancing them against any potential advantages. A counter-example of this requirement would be work of Altınay and Örs [AO21], whose ISE design supports computation of the Ascon S-box: they introduce an instruction which loads five 32-bit inputs $x_i \leftarrow \mathsf{MEM}[\mathsf{GPR}[rs_1] + 4 \cdot i]^4$, applies the S-box to produce outputs r_i from the inputs x_i, then stores five 32-bit outputs $\mathsf{MEM}[\mathsf{GPR}[rs_1] + 4 \cdot i]^4 \leftarrow r_i$, where $0 \leq i < 5$ throughout. These CISC-like semantics contrast sharply with those in the RISC-like RV32I base ISA.

Requirement $\mathcal{R}_2$: minimise number of encoding points. To allow unambiguous decoding, the instruction encoding specified by a given base ISA will imply some space of usable opcodes which each identify an associated instruction: these are often termed encoding points. The space of instruction encoding points is typically structured, e.g., by partitioning it into sub-spaces of "similar" instructions via use of one or more instruction formats. Either way, the space of encoding points must be carefully managed, because, for example, 1) it will often be (very) constrained in terms of size, and 2) decisions about use of it will have a long-term impact with respect to forward-facing extensibility *and* backward-facing compatibility.

The design of an ISE would ideally take this into account, e.g., by minimising the number of encoding points it uses (or consumes).

Requirement $\mathcal{R}_3$: respect existing instruction formats; consider encoding complexity. One could loosely define instruction encoding complexity as a metric including major factors such as the choice of fixed- versus variable-length encoding, plus minor factors such as the number and regularity of instruction formats. Although the base ISA has a significant influence, extension of it via an ISE implies a need to consider instruction encoding complexity in order to avoid various forms of (negative) impact. First, and more obviously, instruction encoding complexity may have an impact on decoding complexity; in turn, this may impact the instruction decoder implementation, e.g., in terms of the associated latency and/or area. For example, latency is not so important for an area-optimised micro-architecture, but area is: it would be problematic if the instruction decoder represented a significant fraction of total area. Likewise, area is not so important for a latency-optimised micro-architecture, which in fact often use multiple, parallel instruction decoders, but latency is: it would be problematic if the instruction decoder represented a bottleneck that limited instruction throughput. Second, but less obviously, instruction encoding complexity may have an impact on the wider micro-architecture. For example, consider a base ISA with a 3-address (2 source, 1 destination) instruction format whose input bandwidth is increased in an ISE which specifies a 4-address (3 source, 1 destination) instruction format. An implementation of the ISE will potentially need to alter 1) the number of read ports the register file must support (assuming reuse of the existing ports in multiple cycles is ruled out), plus 2) any forwarding logic; it is likely that doing so has an impact in terms of area, and plausible the overhead might even dominate that related to associated AFUs.

The design of an ISE would ideally take this into account, e.g., by avoiding additional instruction formats without a clear rationale.

Requirement $\mathcal{R}_4$: minimise additional state; maximise control over said state. At a high-level, additional state specified as part of an ISE implies some overhead, e.g., in terms

of area, because additional storage components (e.g., registers, or memory) will be required to retain it. Beyond this, however, at a low-level, various other technical considerations may also be important. First, there are considerations re. functionality. For example, for application-class cores that support an Operating System (OS) kernel, the additional state may need to be context-switched. Such a situation 1) implies a requirement that the state can be accessed (so is architecturally visible), and 2) implies some overhead in terms of time and space, i.e., to store it into or load it from fields in a relevant Process Control Block (PCB). Second, there are considerations re. security (noting that context-switching *already* has a security dimension: see, e.g., [SP18, GPM23]). For example, additional state may represent an attack target and/or yield information of use to an attacker; one instance of the latter is (micro-)architectural leakage, such as that stemming from the overwrite effect [PV17, Section 3.3] or the neighbour effect [PV17, Section 3.1] within (micro-)architectural registers. Set within this context, a general principle (see, e.g., [GYH18, Section 3]) is that storage components should support a flush or reset mechanism that allows control over the state they contain.

The design of an ISE would ideally take this into account, e.g., by considering whether additional state is required at all (or, whether an alternative is viable), then minimising the amount of and maximising the control over any additional state that *is* required. A counter-example of this requirement would be work of Kuo, Garcia-Herrero, and Maestro [KGM21], whose ISE design supports multiplication in $\mathbb{F}_{2^m}$. they introduce additional state (or "*internal parameters*", to e.g, store the irreducible polynomial) plus instructions to write it, but *not* to read it and so without seeming to consider how it might be context-switched.

Requirement $\mathcal{R}_5$: consider ABI. A base ISA will typically be supplemented by an associated Application Binary Interface (ABI), which one could view as a specification for how resources in the ISA should be used by software. It will, for example, include 1) a function call convention, which is supported by or includes 2) a register convention (i.e., a set of roles for distinguished registers). Stability of and compliance with such an ABI is important, because a wider eco-system of both hardware and software components depend on it. For example, a stable ABI can afford compatibility between software components: it enables callee functions in some library to be invoked by a caller in an application using it, irrespective of the tool-chain used to compile them.

The design of an ISE would ideally take this into account, e.g., by minimising any deviation from or change to the ABI associated with the base ISA. A counter-example of this requirement would be work of Escouteloup et al. [EFLL20], who base a "secure" ISA design (termed RV32S) on RV32I. One recommendation [EFLL20, Recommendation 1] is to "*[t]ag some registers (e.g. `x8` to `x15`) as confidential*" meaning "*[t]hey cannot be used as the source address in load and store instructions*". Doing so affords some positives, e.g., it prevents leakage from memory access based on confidential addresses. However, it also implies some (arguable) negatives, e.g., as written the restriction impacts the RISC-V ABI [RIS22, Section 1.2] in the sense it prevents use of `x8` as a frame pointer.

Requirement $\mathcal{R}_6$: consider privilege model. A base ISA will typically define an associated privilege model, which captures the specification of unprivileged and privileged (e.g., user and kernel) domains. It will, for example, include 1) a set of privilege levels (or modes), and 2) policies around execution of instructions or access to resources for each such level. Note that although unprivileged is a reasonable default for many ISEs, greater care may be important for those which have a security- (cf. Section 3.3) rather than computation- or storage-oriented remit.

The design of an ISE would ideally take this into account, e.g., by minimising any deviation from or change to the privilege model associated with the base ISA, as well as specifying how constituent features are dealt with in terms of it. An example of this

requirement would be work of Saarinen, Newell, and Marshall [SNM22, Section 5.1], who analyse various privilege-related aspects of and decisions relating to the RISC-V entropy source design.

Requirement $\mathcal{R}_7$: consider data-path scalability. Although the base ISA will specify a word size, the concept of word size scalability (cf. PLX [LF05, Section 2]) is sometimes pertinent. For example, the RISC-V *scalar* base ISAs (RVI) include RV32I, RV64I, and RV128I, which relate to 32-, 64-, and 128-bit word sizes respectively; the RISC-V *vector* ISE (RVV) allows implementation-defined parameters such as the vector word size, i.e., `VLEN` (the number of bits in one vector, where `ELEN` ≥ 8) and vector sub-word size, i.e., `ELEN` (the maximum number of bits in one element, or sub-word of a vector, where `VLEN` $\geq$ `ELEN`).

For a given ISE design, it is important to consider whether and how to address this fact. Assuming word size scalability is not ignored, at least two approaches may be viable. First, it may be possible to define many ISEs which apply to different word sizes. The RISC-V Zkne [RIS24, Section 32.3.5] extension includes `aes32esmi` and `aes64esm`, for example, whose semantics are non-uniform across RV32I and RV64I respectively; doing so capitalises on specificity by supporting the most efficient implementation strategy for each word size. Second, it may be possible to define one ISE which applies to, or scales across different word sizes. The RISC-V Zkc [RIS24, Section 32.3.2] extension includes `clmul` and `clmulh`, for example, whose semantics are uniform across RV32I and RV64I respectively; doing so yields a consistent programming model parameterised by the word size.

Requirement $\mathcal{R}_8$: consider instruction fusion. Although the term instruction fusion has an intuitive meaning, more precise definition leads to some subtly. For example, the term macro-op fusion means fusion "between" or inter-instruction; it involves combination of multiple architectural instructions into one micro-architectural micro-op then executed. Likewise, the term micro-op fusion means fusion "inside" or intra-instruction, involving combination of multiple micro-architectural micro-ops stemming from one architectural instruction so they can be executed as one "unit". Given both of the above could be described as dynamic, micro-architectural forms of fusion, one could also define a static, architectural alternative: the ARM "flexible second operand" represents an instance, in the sense it enables semantics such as $\mathsf{GPR}[rd] \leftarrow \mathsf{GPR}[rs_1] \oplus (\mathsf{GPR}[rs_2] \lll \texttt{imm})$ which fuse (pre-)rotation and XOR into rotate-plus-XOR. In fact, one could view many ISE designs as architectural fusion in the sense they define one ISE instruction that, e.g., captures computation which *could* be expressed using multiple instructions from the base ISA.

Whether and how to consider instruction fusion is difficult, because it can depend heavily on other constraints and goals. However, it seems reasonable to suggest that the design of an ISE would ideally do so. For example, Fiskiran and Lee [FL04, Section 3.3] explore three possible ISE designs for carry-less multiplication. Versus use of one architecturally fused instruction, having to execute both `bfmul.lo` and `bfmul.hi` to compute the more- and less-significant w-bit halves of a $(2 \cdot w)$-bit product might seem disadvantageous. However, it could in fact be viewed as advantageous in the sense that it 1) avoids *forcing* the micro-architectures to support an instruction format with 2 destinations (per Section 3.2), but 2) still allows the micro-architectures to apply macro-op fusion *if* doing so is deemed beneficial. Celio et al. [CDPA16, Section IV.D] identify a similar idiom for standard (unsigned) integer multiplication in RV64I; this and other instances are explored further by Singh et al. [SPJR22].

4.1.3 ISE specification

Motivated by their importance as an interface between hardware and software, Reid [Rei19], for example, articulates the need for high quality ISA and so ISE specification. Precision and clarity of said specification are imperative, as is the need for accessibility across

disciplines: the latter is true because "*[s]upporting many different groups leads to improved quality as each group finds different problems in the specification; and, by providing value to each different group, it helps justify the considerable effort required to create a high quality specification of a major processor architecture*". The question is, what approaches exist for doing so?

Based on examples in Section 3, such approaches can be grouped into (roughly) three classes: 1) informal, e.g., descriptive, 2) formal, human-readable, e.g., algorithmic, and 3) formal, machine-readable, e.g., via a Domain-Specific Language (DSL) such as SAIL [ABC+19] or CoreDSL [ODKM+24, KSMGS24]. These classes represent a trade-off of sorts, in the sense they balance challenge (e.g., dependency on tools, technical competence, workload) against quality (e.g., precision, reproducibility, utility). For example, class 2 is the most common class: versus class 1 it is potentially more challenging, but is advantageous because an algorithmic specification of instruction semantics will minimise ambiguity. Class 3 is the least common class, but arguably that which should be aspired to: versus class 2 it is potentially more challenging, but is advantageous because a machine-readable specification can be leveraged, e.g., in support of associated formal verification tasks. In short then, selection of such a class likely depends on understanding the intended audience (i.e., a machine, or parser, and/or a human) and associated use-cases: those may reasonably differ, depending, for example, on whether the context is say a research paper (where clarity, e.g., of semantics, in support of accessibility might favour class 2) or a standard (where formality in support of utility might favour class 3).

4.2 Implementation

In this Section we consider implementation from a hardware-facing perspective, i.e., how the ISE is *realised* in hardware, then from a software-facing perspective, i.e., how the ISE is *utilised* in software. Since both tasks are heavily context-dependent, we adopt a high-level focus on coverage of general points throughout.

4.2.1 Hardware-facing

One could roughly divide this task into at least three related sub-tasks: 1) alter auxiliary micro-architectural components (e.g., special- and general-purpose registers (or files thereof), forwarding logic, etc.) to offer suitable support for ISE-based instructions, 2) alter the instruction decode stage so that ISE-based instructions can be decoded, 3) alter the instruction execution stage so that ISE-based instructions can be executed (e.g., by integrating AFUs alongside an existing ALU). In the same way that the base ISA influences the ISE design, the base micro-architecture influences the ISE implementation and therefore when and how said (sub-)tasks are addressed.

Some micro-architectures have dedicate support for ISEs via some form of interface. Note that Hodjat and Verbauwhede [HV04] study this approach in the context of loosely-coupled cryptographic ISEs; Beyond this, some specific examples include the RoCC interface in RocketCore and CV-X-IF interface in CVA6; some more generic examples include the ARISE [VN07] and SCAIE-V [DOSK22] frameworks (the latter of which surveys RISC-V interfaces more generally). From a positive perspective, such support likely makes the task as a whole easier. For example, Piscopo et al. [PDM+25] compare the trade-offs stemming from various integration approaches (spanning tightly- and loosely-coupled, per Section 2) in support of their Keccak RISC-V Optimized eNgine fOr haShing (KRONOS) design; see also Dolmeta [DMM25], who focus on the Core-V eXtension Interface (CV-X-IF) interface. From a negative perspective, however, it may also constraint the ISE design and/or implementation: use the interface may, for example, impact the instruction encoding[4] or incur overhead, e.g., with respect to latency of instruction execution.

[4] An example would be RoCC [HV04], which uses a R-type RISC-V instruction format but reserves the

4.2.2 Software-facing

The starting point for this task is typically some form of base implementation developed using the base ISA; this may, for example, have been used to inform the ISE identification task. Said base implementation is then *re*developed using the ISE: the question is, how can the latter be supported. First, it is at least plausible to support development using high-level languages and tools, e.g., using the C language and GCC or LLVM compilers (cf. [KSMGS24]). Although the domain-specific nature of ISEs can represent a significant challenge, examples include exposure of the ISE through mechanisms such as intrinsic or built-in functions. However, even where such an approach is possible, security (cf. [SCA18, SLP+24]) and/or efficiency (cf. [CJL+20]) benefits of lower-level cryptographic development are similar whether the ISE or base ISA is used. So, second, one could support development using general-purpose (i.e., ISE-agnostic) or special-purpose (i.e., ISE-aware) low-level tools. Examples of the former include exposure of the ISE through mechanisms such as "rich" assembler macros (e.g., the `.insn` directive for RISC-V); examples of the latter include various work (see, e.g., [MG10, MMG10, BLT15] in relation to AES-NI, plus tools such as SLOTHY [ABKK23] more generally) related to ISE-aware code generation and/or optimisation.

4.3 Verification

With respect to formal verification of computation- and storage-oriented support, ISAs and ISEs present broadly the same challenge: such support can be considered in mainly functional terms, meaning that, perhaps modulo any particularly exotic ISE design, existing verification methodologies associated with the former can likely be reused for the latter. Note that use of a machine-readable specification (cf. Section 4.1.3) can enable or facilitate associated tasks, and that use of open-source tools (see, e.g., [Mar19]) is both viable and can reduce the barrier to entry and reproducibility (which can be especially important when considered in the context of academic work).

With respect to formal verification of security-oriented support, however, the situation is somewhat different. On one hand, the importance of associated tasks is obvious: Uhle, Stolz, and Moradi [USM24] highlight the impact of non- or under-application of appropriate verification, for example. On the other hand, however, the challenge involved is arguably higher (cf. [DXZS16]). Justification of this claim would span numerous factors, but two stand out in particular.

First, some properties can be difficult to formalise, e.g., because they demand doing so in more behavioural terms. For example, properties related to non-discrete forms of information leakage (e.g., power consumption, versus say execution latency) can fall into this category; related methodologies are starting to emerge, such as reconsideration and so redefinition of the hardware and software interface (see, e.g., "contract-like" work such as [BGG+22, HHB24, WMvG+23, MGR24]) to better facilitate (co-)verification. Second, some properties can conflict with or be unsupported by existing verification methodologies. For example, consider any ISE design that uses randomness (e.g., to support a masking countermeasure). Functional verification of such an ISE will require all inputs to an instruction are either 1) perfectly controllable, and/or 2) predictable given a known initial state. This might mean designing the ISE such that randomness is supplied externally (e.g., through a separate instruction, or modelable/drivable interface), or ensuring that the initial "seed" is fully under the control of the developer; another approach might be to support a deterministic "test mode" used only during initial validation (i.e., subsequently disabled, with care taken to avoid it being re-enabled or bypassed). Each such option will impact the security model, so must be considered as part of a rigorous ISE design process. This illustrates the importance of adopting Design for Test (DfT) and Design for

`func3` field to specify whether GPR[`rs1`] and GPR[`rs2`] are read, and whether GPR[`rd`] is written.

Verification (DfV), where design decisions are made to render test and verification (be it for functional, security, or both) easier. A parallel may be drawn to cryptography more generally, where primitive design might employ a specific approach or component in order to make an associated security proof easier.

4.4 Evaluation

One could describe evaluation as having (at least) two goals: 1) to practically validate outcomes of verification, and 2) to provide evidence in support of decision making around fitness-for-purpose, comparison with alternatives, etc. Note that (subtle) framing of the latter around delivery of evidence, versus a definitive conclusion, is intentional. That is, an evaluation which maximises the volume and quality of evidence is arguably more useful, because it permits more informed, effective decisions based, e.g., on the context, pertinent trade-offs, etc.

As a starting point, one might analytically evaluate the (abstract) design: although it can be difficult to quantify (non-)compliance with requirements, considering instances such as those explored in Section 4.1 would be one example. Beyond this, one might experimentally evaluate the (concrete) implementation, e.g., through data related to pertinent metrics:

Concept 17. *An ISE evaluation metric might be described as* **primary** *or* **secondary***: the latter essentially combine or reflect one or more primary metrics.*

Concept 18. *An ISE evaluation metric might be measured at an* **instruction-level**, **kernel-level**, *or* **system-level***: these terms essentially differentiate between a lower-level or more local focus, versus a higher-level or more global focus.*

Examples: primary, instruction- and kernel-level. From an efficiency-oriented perspective, standard metrics aligned with design (D), hardware (H), and software (S) aspects can be readily identified: a non-exhaustive set of examples include

1. (D) encoding efficiency, measured, e.g., in number of encoding points,
2. (H) area overhead, measured, e.g., in Gate Equivalents (GE),
3. (H) impact on critical path, measured, e.g., in change in maximum clock frequency,
4. (S) memory footprint (and/or code density), measured, e.g., in bytes,
5. (S) memory accesses, measured, e.g., in number of accesses,
6. (S) execution latency, measured, e.g., in number of cycles,
7. (S) execution throughput: measured, e.g., in operations per cycle.

Note that use of memory can be decomposed into sub-metrics based on 1) data (where access means loads and stores) and instruction (where access means fetches), plus 2) static (e.g., global data) and dynamic (e.g., stack-based data), any combination of which can be useful.

From a security-oriented perspective, however, the correct approach is less clear: it depends on the context, and, for example the security properties in question. Modulo cases where the ISE has access to some privileged resource, security evaluation of an ISE-supported implementation can follow a similar approach to that for a (pure) software implementation. Note, however, that dedicated approaches (cf. CISELeaks by Jayasena, Bachmann, and Mishra [JBM24]) may be relevant in specific cases or for specific properties.

Examples: primary, system-level. Bernstein[5] observes that research on ISE design will often under-emphasise "*the CPU designer's perspective: the new instructions cost chip area, and are competing with many other suggestions for productive ways to use the same chip*

[5] https://blog.cr.yp.to/20140517-insns.html

area". Consider the example of TCP/IP-based communication between parties, secured using TLS [Res18]. Within the presentation layer, components in the agreed cipher suite might be invoked on a per-packet (for the record protocol, so, e.g., AES) or even a per-session (for the handshake protocol, so, e.g., RSA) basis, with that communication itself set alongside computation, i.e., the application layer, dependent on it. Put simply then, Amdahl's law will apply: the impact of an ISE at a system-level will be limited by how often it is utilised, a fact that assessment of viability (e.g., whether the cost is "worth" any associated improvement in performance) may need to consider.

Examples: secondary. Some secondary metrics can be framed as *explicitly* combining one or more primary metrics. For example, Paar [Paa02, Point 1] advises to "*optimize the performance-cost product*" for cryptographic implementation tasks in general. For the more specific task of ISE design, an instance of said advice might therefore focus on "*improvement in execution latency per GE of additional area*" for example.

Some secondary metrics can be framed as *implicitly* combining one or more primary metrics. For example, use of an ISE often means a more compact implementation: there are fewer (static) instructions in, and fewer (dynamic) instruction fetches from memory. These primary metrics may also be reflected as a contributing factor to the secondary metric of energy consumption (and so energy efficiency).

Pit-falls.

- It can be attractive to quote the area of an isolated AFU. A (potential) pit-fall of doing so is that overhead associated with integration of the AFU is ignored. If the AFU itself is particularly lightweight, for example, then interfacing (e.g., multiplexers to integrate the AFU into an ALU-like data-path) or even decoding logic can be significant.
- It can be attractive to quote area as an overhead: if the base micro-architecture (i.e., including the ISE) area is x and the extended micro-architecture (i.e., excluding the ISE) area is y, then one might, for example, quote the factor y/x. A (potential) pit-fall of doing so is that if x is (perhaps artificially) large, then both $y - x$ and y/x are likely to be small. Put simply, the absolute area associated with an ISE implementation might be obscured by use of relative area as the metric. There is no single correct solution to this pit-fall, since it depends on the context, and so, e.g., what is or is not relevant when defining the base micro-architecture. However, one approach would be to focus on and so include the core resources involved in executing instructions (e.g., pipeline), but exclude any uncore resources (e.g., caches, peripherals).
- Using an FPGA-based fabric to prototype some design and implementation is a reasonable, accepted approach both in the case of general hardware design *and*, therefore, the specific case of cryptographic ISE design. Obvious benefits stem from reconfigurabilty, which, e.g., allows more efficient design exploration (because development iterations are less time and resource intensive). A (potential) pit-fall of doing so, however, is that an eventual deployment of the ISE *might* instead be based on an ASIC-based fabric. Various studies (see, e.g., [KR07, KR10]) have explored the challenge of extrapolation from FPGA- into ASIC-based results: doing so accurately for efficiency-oriented metrics can already be difficult, with security-oriented metrics arguably even more so. In the best case, useful steps from FPGA- towards ASIC-based results are possible without significant overhead. One could, for example, use the open-source Yosys[6] synthesis tool to produce results using a generic technology library based on NAND2 gate equivalents. In the worst case, the gap between FPGA- to ASIC-based results and implications from it should clearly temper any conclusions.

[6] https://yosyshq.net/yosys

- A simplistic "drop-in" approach to use of an ISE in software may ignore opportunities for optimisation, and therefore lead to inaccurate evaluation: ideally, one would at least reconsider and make changes to the implementation strategy where/when appropriate. For example, relative to use of the base ISA, an equivalent ISE-based loop body will typically include fewer instructions. The loop overhead (e.g., test and update of a loop counter) stemming from iteration over is therefore likely to be more prominent; reconsidering any decisions related to use of loop unrolling (plus the unrolling factor used) may be important.

5 Discussion of emerging trends, open challenges, etc.

In this Section, we discuss cryptographic ISEs from a more future-facing perspective: based on coverage of existing work in Section 3 and Section 4, for example, the goal is to identify and discuss related concepts, emerging trends, and open challenges (or opportunities). Accurate futurology is notoriously difficult, particular with regards to technology, meaning we frame the content more as a non-exhaustive set of (evidenced) observations and opinions. We claim that doing so is still useful, however, in the sense it can potentially guide future work and perhaps the field as a whole.

Toward "full-stack" co-design. Paar [Paa02, Point 4] advises to "*focus on developing algorithms for software and hardware which take the nature of the target platform into account*". Modulo the challenge and potential disadvantages[7] (or pit-falls) of doing so, it seems clear that such an approach will help to maximise the efficacy of associated outcomes; based on evidence from (i.e., candidates submitted to) the National Institute of Standards and Technology (NIST) LWC and Post-Quantum Cryptography (PQC) standardisation processes, for example, one could in fact argue it now represents the norm. Although exceptions exist (see, e.g., [GIP+06]), however, the same approach is less common when framed within the context of ISE design specifically. For example, neither the NIST LWC and PQC standardisation processes explicitly considered or supported ISEs. We argue this is at odds with the fact that, based on the AES standardisation process, standardised cryptography of sufficient value *will* (eventually) be supported by ISAs, e.g., via ISEs: it seems fair to claim that considering ISEs as part of future standardisation processes might help maximise the quality of their eventual outcome.

Framed against meta guidelines such as [NIS16], how to do so is, of course, a broad and difficult challenge. There *are* selected instances of the topic appearing, e.g., within interim standardisation reports: within the context of LWC, for example, [CGM+22] is cited by [TMC+23]. However, we argue one could maximise the technical merit of outcomes via overt consideration within initial standardisation calls. First, one could argue that ISEs could be reflected within selection of evaluation criteria. For example:

- The NIST PQC call[8] cites the characteristic "*can be parallelized to achieve higher performance*", to which one might plausibly add "*can achieve higher performance with suitable changes to (or extensions of) the instruction set*" or similar.
- The NIST LWC call[9] cites the metric "*suitability for hardware and software implementations*", to which one might plausibly add "*implementations based on hardware/software co-design*". or similar.

[7] An example would be *over*-specialisation of either an algorithm or parameterisation of it to one ISA or micro-architecture, based on features in it. Scott [Sco07] argues that selection of irreducible polynomials used to define $\mathbb{F}_{2^m}$ fall into this category: the "best" selection depends on the ISA, with standardised selections potentially biased, e.g., by features such as the ARM "flexible second operand".

[8] See https://csrc.nist.gov/projects/post-quantum-cryptography.

[9] See https://csrc.nist.gov/projects/lightweight-cryptography.

Second, one could argue that ISEs could be supported within selection of evaluation platforms. For example:

- The NIST PQC call originally defined the "*NIST PQC Reference Platform* as "*an Intel x64 running Windows or Linux and supporting the GCC compiler*" noting "*may also perform efficiency testing using additional platforms*"; they later cited[10] Cortex-M4 (motivated, e.g., by pqm4 [KKPY24]) and others as of interest; to which one might plausibly add some form of soft-core (which can, e.g., by synthesised for the same Field Programmable Gate Array (FPGA) supported for hardware implementations) which can support exploration of ISEs.
- The NIST PQC call (in line with most others) demands submission of a reference implementation whose goal is to "*promote understanding of how the submitted algorithm may be implemented*". One could promote a programming style where instruction-like operations are specified using, e.g., intrinsic-like functions rather than language-supported operators, to demonstrate where and how ISE-based support might be provided.

Provision and use of mechanisms for ISE specification. Katz and Lindell [KL21] summarise modern cryptography as being "*distinguished from classical cryptography by its emphasis on definitions, precise assumptions, and rigorous proofs of security*". In the same way, it seems reasonable to claim that a transition from classical to modern cryptographic engineering has normalised machine-checked proofs for both functional *and* behavioural properties of both hardware *and* software implementation.

Based on Section 3, however, said transition seems much less evident within ISE design. For example, our survey identified few instances of (academic) research that included a formal, machine-readable ISE specification. Pitching such an approach as the ideal, longer-term norm, one might try to assess why this is by asking how easy is it to use such an approach; how reasonable is it to model an ISE versus an entire ISA; how optimised is the language to general-purpose versus special-purpose architectural features; how easy is it to use the resulting model for *other* use-cases and tools; how reasonable is it to present the model in a research paper, e.g., as human-readable LaTeX? We argue that addressing these (and other) questions, e.g., finding ways to support and incentivise use of machine-readable specifications, is important: doing so will allow the associated advantages to be realised.

Beyond micro-architecturally "simple" base cores. Within Section 3, and although counter-examples exist, it seems reasonable to say that the vast majority of (academic) research is focused on embedded- rather than application-class or even high-performance base cores. For example, such a base core might employ a scalar (versus super-scalar), in-order (versus out-of-order) pipeline.

On one hand, this approach makes sense: such base cores are more readily available, simpler, and so easier to work with. On the other hand, however, one could argue they represent too limited a scope relative to the design space as a whole, and, e.g., establish a "feedback loop" in terms of ISE design approaches. As such, and for (at least) two reasons, a longer-term shift in focus seems important. First, at least considering the requirements and constraints of more complex base cores is likely to yield a more robust and applicable (i.e., more easily, more widely adoptable) ISE designs; beyond this, (research) opportunities as well as challenges might stem from analysing and designing ISEs for such cores. Second, doing so is plausible now in a way it was not 5 to 10 years ago: depending on the definition of complex, initiatives such as RISC-V have, for example, led to well supported instances such as CVA6 (née Ariane [ZB19]) and BOOM [ZKGA20] being available.

[10] See https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/_0mDoyry1Ao/m/Tt7yHpjSDgAJ.

Beyond traditional micro-processor ISEs. Accurately predicting technology-related trends is notoriously difficult, although perhaps easier for specific technologies. This has led various authors (see, e.g., [GGPY89, Yu96, BC11]) to make such predictions about trends in micro-processor design, which span lower- (e.g., transistors) through to higher-level (e.g., micro-architectures) aspects. Fixing an even more specific remit, we claim that (at least) two trends are evident in cryptographic ISE design. First, and in the shorter-term, the relevance of computational fabrics other than micro-processors has been mirrored by activity related to ISE design. For example:

- Adams et al. [AGTK21] present a cryptographic ISEs design for a specific type of Graphics Processing Unit (GPU).
- Mentens, Charbon, and Regazzoni [MCR18] study the design of "cryptography-friendly" FPGA cells: drawing an analogy between instruction and cell, one might view their work as ISE-like in the sense it extends support by an FPGA for some domain-specific workload.
- Ahmed and Tsoutsos [AT24] consider ISEs for PulpFHE, a micro-processor design (based on Juliet [GMT24]) offering native support for Fully Homomorphic Encryption (FHE).

Second, and in the longer-term, there is a general trend towards computational fabrics supporting reconfigurability. For example:

- More coarse-grained instances include a variety of reconfigurable micro-processors, such as Garp [HW97], ConCISe [KBH99], PipeWrench [TG99], MorphoSys [SLL+00], Chimaera [HFHK03], and COBRA [EP05].
- More fine-grained instances include reconfigurable resources within such a micro-processor, e.g., AFU realised using embedded Field Programmable Gate Array (eFPGA). Dao et al. [DAHK20] present a FlexBex, for example, a concrete RISC-V compliant core which supports reconfigurable ISE.
- Kumar and Paar [KP04] consider a commercial platform based on FPSLIC (essentially an AVR micro-controller plus an FPGA), using it to realise a co-processor for arithmetic in $\mathbb{F}_{2^{163}}$.
- Grabher et al. [GGH+11] consider an experimental platform based on SPARC (essentially a LEON3 hosted on an FPGA, plus a reconfigurable region of said FPGA), using it to realise a range of cryptographic ISEs.

Advantages and disadvantages of implementation diversity. Viewed from a positive perspective, an ISE-based implementation will often represent a more direct (or natural) mapping from the associated specification: this fact stems, e.g., from provision of functionality via the ISE which is lacking in the base ISA, and might also have implications for verification. From a negative perspective, however, such implementations are also less diverse because said mapping is constrained, or even *implied* by the ISE. Whether or not implementation diversity is a relevant metric, it at least seems fair to say the associated implications have prompted only limited exploration. For example, *if* ISE-based implementations are less diverse then it seems plausible they could 1) be easier to identify, in either a static (e.g., in a machine code image, per [MMW21]) or a dynamic (e.g., in a power consumption trace, per [TBW+21]) setting, and 2) lead to more accurate, portable profiles with respect to associated side-channel attacks (e.g., those using templates, per [CRR02]).

6 Conclusion

In general terms, one might expect a SoK-like paper on some topic to 1) survey, analyse, and evaluate associated work spanning historical development plus the state-of-the-art (at the time of writing), 2) systematise said work, e.g., via taxonimisation of concepts and

results, and 3) provide insight with respect to said work and the broader field, e.g., by identifying trends, plus open research directions and challenges. In this paper, we focused on the specific topic of cryptographic ISEs. We claim that the body of associated work captures an important point in what is a large, complex design space of implementation options for cryptography; this claim is evidenced by a rich history, *and* a vast corpus of hardware and software artefacts and associated literature (e.g., per our 308-entry bibliography, and including notable theses such as [Shi04, Fis05, Hil08, Til08, Man11, Raw16, Nis21, Nis21, Fri22, Pet25]). Within the paper, Section 2 focused on point 2), e.g., by systematising associated concepts and terminology. Section 3 focused on points 1) and 2), e.g., by surveying and systematising associated work. Section 4 and Section 5 focused on point 3), e.g., by surveying and systematising development processes which produced such work.

Acknowledgements

This work has been supported in part by EPSRC via grant EP/R012288/1 under the RISE (http://www.ukrise.org) programme, Innovate UK via project 10065634 (SCHEME: Safety Critical Harsh Environment Micro-processing Evolution), and the National Natural Science Foundation of China via grant 62502275.

References

[ABC+19] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K.E. Gray, R.M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell. ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS. In *Principles of Programming Languages (POPL)*, pages 71:1–71:31, 2019. https://doi.org/10.1145/3290384.

[ABKK23] A. Abdulrahman, H. Becker, M.J. Kannwischer, and F. Klein. Fast and clean: Auditable high-performance assembly via constraint solving. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(1):87–132, 2023. https://doi.org/10.46586/tches.v2024.i1.87-132.

[Aca97] T. Acar. *High-Speed Algorithms & Architectures for Number Theoretic Cryptosystems*. PhD thesis, Department of Electrical & Computer Engineering, Oregon State University, 1997. https://search.library.oregonstate.edu/permalink/01ALLIANCE_OSU/1c3q896/alma99101945101865.

[AEL+20] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri. ISA extensions for finite field arithmetic: Accelerating Kyber and NewHope on RISC-V. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020(3):219–242, 2020. https://doi.org/10.13154/tches.v2020.i3.219-242.

[AGTK21] A. Adams, P. Gupta, B. Tine, and H. Kim. Cryptography acceleration in a RISC-V GPGPU. In *Computer Architecture Research with RISC-V (CARRV)*, 2021. https://carrv.github.io/2021.

[AMD17] AMD. AMD random number generator, 2017. https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/white-papers/amd-random-number-generator.pdf.

[AO21] Ö. Altınay and B. Örs. Instruction extension of RV32I and GCC back end for Ascon lightweight cryptography algorithm. In *International Conference on Omni-Layer Intelligent Systems (COINS)*, pages 1–6, 2021. https://doi.org/10.1109/COINS51742.2021.9524190.

[AOP+25] A. Abdulrahman, F. Oberhansl, H.N.H. Pham, J. Philipoom, P. Schwabe, T. Stelzer, and A. Zankl. Towards ML-KEM & ML-DSA on OpenTitan. In *IEEE Symposium on Security and Privacy (SP)*, pages 1–19, 2025. https://doi.org/10.1109/SP61157.2025.00220.

[API03] K. Atasu, L. Pozzi, and P. Ienne. Automatic application-specific instruction-set extensions under microarchitectural constraints. *International Journal of Parallel Programming*, 31(6):411–428, 2003. https://doi.org/10.1023/B:IJPP.0000004508.14594.b9.

[APRJ11] A. Arora, S. Parameswaran, R.G. Ragel, and D. Jayasinghe. A hardware/software countermeasure and a testing framework for cache based side channel attacks. In *Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1005–1014, 2011. https://doi.org/10.1109/TrustCom.2011.138.

[Arm20] Arm architecture reference manual: Armv8, for Armv8-A architecture profile. Technical report, 2020. https://static.docs.arm.com/ddi0487/fa/DDI0487F_a_armv8_arm.pdf.

[AT24] O. Ahmed and N.G. Tsoutsos. PulpFHE: Complex instruction set extensions for FHE processors. Cryptology ePrint Archive, Paper 2024/1315, 2024. https://eprint.iacr.org/2024/1315.

[BBD+05] P. Biswas, S. Banerjee, N. Dutt, L. Pozzi, and P. Ienne. ISEGEN: generation of high-quality instruction set extensions by iterative improvement. In *Design, Automation, and Test in Europe (DATE)*, pages 1246–1251, 2005. https://doi.org/10.1109/DATE.2005.191.

[BBFR06] G. Bertoni, L. Breveglieri, R. Farina, and F. Regazzoni. Speeding up AES by extending a 32-bit processor instruction set. In *Application-Specific Systems, Architectures and Processors (ASAP)*, pages 275–282, 2006. https://doi.org/10.1109/ASAP.2006.62.

[BBGR09] R. Benadjila, O. Billet, S. Gueron, and M.J.B. Robshaw. The Intel AES instructions set and the SHA-3 candidates. In *Advances in Cryptology (ASIACRYPT)*, LNCS 5912, pages 162–178. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-10366-7_10.

[BC11] S. Borkar and A.A. Chien. The future of microprocessors. *Communications of the ACM (CACM)*, 54(5):67–77, 2011. https://doi.org/10.1145/1941487.1941507.

[BCA+04] P. Biswas, V. Choudhary, K. Atasu, L. Pozzi, P. Ienne, and N. Dutt. Introduction of local memory elements in instruction set extensions. In *Design Automation Conference (DAC)*, pages 729–734, 2004. https://doi.org/10.1145/996566.996765.

[BCM+23] S. Belaïd, G. Cassiers, C. Mutschler, M. Rivain, T. Roche, F.-X. Standaert, and A.R. Taleb. A methodology to achieve provable side-channel security in real-world implementations. Cryptology ePrint Archive, Paper 2023/1198, 2023. https://eprint.iacr.org/2023/1198.

[BDPI07] P. Biswas, N. Dutt, L. Pozzi, and P. Ienne. Introduction of architecturally visible storage in instruction set extensions. *IEEE Transactions on Computers*, 26(3):435–446, 2007. https://doi.org/10.1109/TCAD.2006.890582.

[BEM+15] N. Benhadjyoussef, W. Elhadjyoussef, M. Machhout, R. Tourki, and K. Torki. Enhancing a 32-bit processor core with efficient cryptographic instructions. *Journal of Circuits, Systems and Computers*, 24(10), 2015. https://doi.org/10.1142/S0218126615501583.

[BGG+22] R. Bloem, B. Gigerl, M. Gourjon, V. Hadzic, S. Mangard, and R. Primas. Power contracts: Provably complete power leakage models for processors. In *Computer and Communications Security (CCS)*, pages 381–395, 2022. https://doi.org/10.1145/3548606.3560600.

[BGHZ11] G. Barthe, B. Grégoire, S. Heraud, and S. Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *Advances in Cryptology (CRYPTO)*, LNCS 6841, pages 71–90. Springer-Verlag, 2011. https://doi.org/10.1007/978-3-642-22792-9_5.

[BGM03] I. Branovic, R. Giorgi, and E. Martinelli. A workload characterization of elliptic curve cryptography methods in embedded environments. In *Workshop on MEmory Performance: DEaling with Applications, Systems and Architecture (MEDEA)*, pages 27–34, 2003. https://doi.org/10.1145/1152923.1024299.

[BGM09] S. Bartolini, R. Giorgi, and E. Martinelli. Instruction set extensions for cryptographic applications. In Ç.K. Koç, editor, *Cryptographic Engineering*, chapter 9, pages 191–233. Springer, 2009. https://doi.org/10.1007/978-0-387-71817-0_9.

[BHO04] R. Buchty, N. Heintze, and D. Oliva. Cryptonite - a programmable crypto processor architecture for high-bandwidth applications. In *Architecture of Computing Systems (ARCS)*, LNCS 2981, pages 184–198. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-24714-2_15.

[BKL+07] A. Bogdanov, L.R. Knudsen, G. Leander, C. Paar, A. Poschmann, M.J.B. Robshaw, Y. Seurin, and C. Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 4727, pages 450–466. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-74735-2_31.

[BLCN20] M. Bie, W. Li, T. Chen, and L. Nan. Design and implementation of cryptographic instruction set. In *IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT)*, pages 1–3, 2020. https://doi.org/10.1109/ICSICT49897.2020.9278206.

[BLT15] A. Bogdanov, M.M. Lauridsen, and E. Tischhauser. Comb to pipeline: Fast software encryption revisited. In *Fast Software Encryption (FSE)*, LNCS 9054, pages 150–171. Springer-Verlag, 2015. https://doi.org/10.1007/978-3-662-48116-5_8.

[BMA00] J. Burke, J. McDonald, and T. Austin. Architectural support for fast symmetric-key cryptography. In *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 178–189, 2000. https://doi.org/10.1145/378993.379238.

[BMT16] W. Burleson, O. Mutlu, and M. Tiwari. Who is the major threat to tomorrow's security? you, the hardware designer. In *Design Automation Conference (DAC)*, pages 145:1–145:5, 2016. https://doi.org/10.1145/2897937.2905022.

[BOS11] J.W. Bos, O. Özen, and M. Stam. Efficient hashing using the AES instruction set. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 6917, pages 507–522. Springer-Verlag, 2011. https://doi.org/10.1007/978-3-642-23951-9_33.

[BSMH25] A. Bolat, S. Sezer, K. McLaughlin, and H. Hui. Microarchitecture design and benchmarking of custom SHA-3 instruction for RISC-V. *cs.AR*, 2508.20653, 2025. https://arxiv.org/abs/2508.20653.

[BSP13] P. Bhagyasree, C. Silpa, and M.J.C. Prasad. A novel RISC processor with crypto specific instruction set. *International Journal of Soft Computing and Engineering (IJSCE)*, 3(5):124–128, 2013.

[BSS+13] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers. The SIMON and SPECK families of lightweight block ciphers. Cryptology ePrint Archive, Report 2013/404, 2013. https://eprint.iacr.org/2013/404.

[BV14] A. Bosselaers and F. Vercauteren. YAES. Technical report, 2014. https://competitions.cr.yp.to/round1/yaesv2.pdf.

[BVIB12] A.G. Bayrak, N. Velickovic, P. Ienne, and W. Burleson. An architecture-independent instruction shuffler to protect against side-channel attacks. *ACM Transactions on Architecture and Code Optimization (TACO)*, 8(4):20:1–20:19, 2012. https://doi.org/10.1145/2086696.2086699.

[Cam80] M. Campbell-Kelly. Programming the Mark I: Early programming activity at the University of Manchester. *Annals of the History of Computing*, 2(2):130–168, 1980. https://doi.org/10.1109/MAHC.1980.10018.

[CB23] S. Cui and J. Balasch. Efficient software masking of AES through instruction set extensions. In *Design, Automation, and Test in Europe (DATE)*, pages 1–6, 2023. https://doi.org/10.23919/DATE56975.2023.10137150.

[CBG12] J.H.-F. Constantin, A.P. Burg, and F.K. Gürkaynak. Instruction set extensions for cryptographic hash functions on a microcontroller architecture. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 117–124, 2012. https://doi.org/10.1109/ASAP.2012.13.

[CCM97] A.D. Carlson, R.W. Castelino, and R.O. Mueller. Multimedia extensions for a 550-MHz RISC microprocessor. *Journal of Solid-State Circuits*, 32(11):1618–1624, 1997. https://doi.org/10.1109/4.641681.

[CDPA16] C. Celio, P. Dabbelt, D.A. Patterson, and K. Asanović. The renewed case for the reduced instruction set computer: Avoiding ISA bloat with macro-op fusion for RISC-V. *cs.CR*, 1607.02318, 2016. https://arxiv.org/abs/1607.02318.

[CFG+24] H. Cheng, G. Fotiadis, J. Großschädl, D. Page, T.H. Pham, and P.Y.A. Ryan. RISC-V instruction set extensions for multi-precision integer arithmetic. In *Design Automation Conference (DAC)*, pages 329:1–329:6, 2024. https://doi.org/10.1145/3649329.3657347.

[CGM+22] H. Cheng, J. Großschädl, B. Marshall, D. Page, and T.H. Pham. RISC-V instruction set extensions for lightweight symmetric cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2023(1):193–237, 2022. https://doi.org/10.46586/tches.v2023.i1.193-237.

[CGM+24] H. Cheng, J. Großschädl, B. Marshall, D. Page, and M-.J.O. Saarinen. SoK: Instruction set extensions for cryptographers. Cryptology ePrint Archive, Paper 2024/1323, 2024. https://eprint.iacr.org/2024/1323.

[CGMM15] M. Cazorla, S. Gourgeon, K. Marquet, and M. Minier. Survey and benchmark of lightweight block ciphers for MSP430 16-Bit microcontroller. *Security and Communication Networks*, 8(18):3564–3579, 2015. https://doi.org/10.1002/sec.1281.

[Chi24] D. Chisnall. How to design an ISA: The popularity of RISC-V has led many to try designing instruction sets. *ACM Queue*, 21(4):27–46, 2024. https://doi.org/10.1145/3639445.

[CHW00] T.J. Callahan, J.R. Hauser, and J. Wawrzynek. The Garp architecture and C compiler. *IEEE Computer*, 33(4):62–69, 2000. https://doi.org/10.1109/2.839323.

[CJL+20] F. Campos, L. Jellema, M. Lemmen, L. Müller, D. Sprenkels, and B. Viguier. Assembly or optimized C for lightweight cryptography on RISC-V? In *Cryptology and Network Security (CANS)*, LNCS 12579, pages 526–545. Springer-Verlag, 2020. https://doi.org/10.1007/978-3-030-65411-5_26.

[CLB+25] S. Cui, G. Luo, J. Bao, J. Balasch, and I. Verbauwhede. Extending and accelerating inner product masking with fault detection via instruction set extension. Cryptology ePrint Archive, Paper 2025/2165, 2025. https://eprint.iacr.org/2025/2165.

[CLLG11] J.K.-T. Chang, C. Liu, S. Liu, and J.-L. Gaudiot. Workload characterization of cryptography algorithms for hardware acceleration. In *ACM/SPEC International Conference on Performance Engineering (ICPE)*, pages 381–390, 2011. https://doi.org/10.1145/1958746.1958800.

[CMM13] M. Cazorla, K. Marquet, and M. Minier. Survey and benchmark of lightweight block ciphers for wireless sensor networks. In *International Conference on Security and Cryptography (SECRYPT)*, pages 1–6, 2013.

[CP20] L. Choquin and F. Piry. Arm custom instructions: Enabling innovation and greater flexibility on Arm. Technical report, Arm Ltd., 2020. https://www.arm.com/why-arm/technologies/custom-instructions.

[CPW24] H. Cheng, D. Page, and W. Wang. eLIMInate: a Leakage-focused ISE for Masked Implementation. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(2):329–358, 2024. https://doi.org/10.46586/tches.v2024.i2.329-358.

[CRR02] S. Chari, J.R. Rao, and P. Rohatgi. Template attacks. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 2523, pages 13–28. Springer-Verlag, 2002. https://doi.org/10.1007/3-540-36400-5_3.

[CZQ+24] C. Chen, X. Zhang, K. Qin, T. Wang, Y. Shi, T. Huang, C. Zhang, and D. Gu. RISecure-PUF: Multipurpose PUF-driven security extensions with lookaside buffer in RISC-V. *cs.CR*, 2411.14025, 2024. https://arxiv.org/abs/2411.14025.

[DAHK20] N. Dao, A. Attwood, B. Healy, and D. Koch. FlexBex: A RISC-V with a reconfigurable instruction extension. In *International Conference on Field-Programmable Technology (ICFPT)*, pages 190–195, 2020. https://doi.org/10.1109/ICFPT51103.2020.00034.

[DB18] C. Dunham and J. Beard. This architecture tastes like microarchitecture. In *Workshop on Pioneering Processor Paradigms (WP3)*, 2018. http://www.jonathanbeard.io/pdf/db18a.pdf.

[DBB+23] M. De Kremer, M. Brohet, S. Banik, R. Avanzi, and F. Regazzoni. Resource-constrained encryption: Extending Ibex with a QARMA hardware accelerator. In *Application-specific Systems, Architectures and Processors (ASAP)*, pages 147–155, 2023. https://doi.org/10.1109/ASAP57973.2023.00034.

[DDHS00] K. Diefendorff, P. Dubey, R. Hochsprung, and H. Scales. AltiVec extension to PowerPC accelerates media processing. *IEEE Micro*, 20(2):85–95, 2000. https://doi.org/10.1109/40.848475.

[DEWW01] G. De Micheli, R. Ernst, W. Wolf, and M. Wolf, editors. *Readings in Hardware/Software Co-Design*. Systems on Silicon. Morgan Kaufmann, 2001. https://doi.org/10.1016/B978-1-55860-702-6.X5000-5.

[DGK19] N. Drucker, S. Gueron, and V. Krasnov. Making AES great again: The forthcoming vectorized AES instruction. In *Information Technology New Generations (ITNG)*, AISC 800, pages 37–41. Springer-Verlag, 2019. https://doi.org/10.1007/978-3-030-14070-0_6.

[DLK+19] D. Dinu, Y. Le Corre, D. Khovratovich, L. Perrin, J. Großschädl, and A. Biryukov. Triathlon of lightweight block ciphers for the internet of things. *Journal of Cryptographic Engineering (JCEN)*, 9(3):283–302, 2019. https://doi.org/10.1007/s13389-018-0193-x.

[DMM25] A. Dolmeta, M. Martina, and G. Masera. Exploring the new CV-X-IF interface to customize RISC-V instruction sets: A case of study in cryptography. In *International Conference on Applications in Electronics Pervading Industry, Environment and Society*, LNEE 1369, pages 39–47. Springer-Verlag, 2025. https://doi.org/10.1007/978-3-031-84100-2_5.

[DOSK22] M. Damian, J. Oppermann, C. Spang, and A. Koch. SCAIE-V: An open-source SCAlable interface for ISA Extensions for RISC-V processors. In *Design Automation Conference (DAC)*, pages 169–174, 2022. https://doi.org/10.1145/3489517.3530432.

[DPM+25] A. Dolmeta, V. Piscopo, G. Masera, M. Martina, and M. Hutter. HORCRUX - a lightweight PQC-RISC-v eXtension architecture. Cryptology ePrint Archive, Paper 2025/1934, 2025. https://eprint.iacr.org/2025/1934.

[DR02] J. Daemen and V. Rijmen. *The Design of Rijndael*. Springer, 2002. https://doi.org/10.1007/978-3-662-04722-4.

[DR14] K.L. Divya and S.H. Rao. Implementation of Cryptographic Risc Processor (CRISC). *International Journal of Innovative Science, Engineering & Technology (IJISET)*, 1(10):358–363, 2014.

[DXZS16] O. Demir, W. Xiong, F. Zaghloul, and J. Szefer. Survey of approaches for security verification of hardware/software systems. Cryptology ePrint Archive, Report 2016/846, 2016. https://eprint.iacr.org/2016/846.

[EAD+22] W. El Hadj Youssef, A. Abdelli, F. Dridi, R. Brahim, and M. Machhout. An efficient lightweight cryptographic instructions set extension for IoT device security. *Security and Communication Networks*, 2022, 2022. https://doi.org/10.1155/2022/9709601.

[EFLL20] M. Escouteloup, J.J.A. Fournier, J.-L. Lanet, and R. Lashermes. Recommendations for a radically secure ISA. In *Computer Architecture Research with RISC-V (CARRV)*, 2020. https://carrv.github.io/2020.

[EKP+13] S. Engels, E.B. Kavun, C. Paar, T. Yalçin, and H. Mihajloska. A nonlinear/linear instruction set extension for lightweight ciphers. In *IEEE Symposium on Computer Arithmetic (ARITH)*, pages 67–75, 2013. https://doi.org/10.1109/ARITH.2013.36.

[Elb07] A.J. Elbirt. Fast and efficient implementation of AES via instruction set extensions. In *Advanced Information Networking and Applications Workshops (AINAW)*, pages 396–403, 2007. https://doi.org/10.1109/AINAW.2007.182.

[Elb08] A.J. Elbirt. Accelerated AES implementations via generalized instruction set extensions. *Journal of Computer Security*, 16(3):265–288, 2008. https://doi.org/10.3233/jcs-2008-16302.

[EMOC20] G.H. Eisenkraemer, F.G. Moraes, L.L. de Oliveira, and E. Carara. Lightweight cryptographic instruction set extension on Xtensa processor. In *IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2020. https://doi.org/10.1109/ISCAS45731.2020.9180579.

[EP05] A. Elbirt and C. Paar. An instruction-level distributed processor for symmetric-key cryptography. *IEEE Transactions on Parallel and Distributed Systems*, 16(5):468–480, 2005. https://doi.org/10.1109/TPDS.2005.51.

[FBR+21] T. Fritzmann, M. Van Beirendonck, D.B. Roy, P. Karl, T. Schamberger, I. Verbauwhede, and G. Sigl. Masked accelerators and instruction set extensions for post-quantum cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2022(1):414–460, 2021. https://doi.org/10.46586/tches.v2022.i1.414-460.

[FHD22] M. Funck, V. Herdt, and R. Drechsler. Virtual prototype driven design, implementation and evaluation of RISC-V instruction set extension. In *International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, pages 14–19, 2022. https://doi.org/10.1109/DDECS54261.2022.9770108.

[Fis05] A.M. Fiskiran. *Instruction set architecture for accelerating cryptographic processing in wireless computing devices*. PhD thesis, Princeton University, 2005.

[FL01] A.M. Fiskiran and R.B. Lee. Performance impact of addressing modes on encryption algorithms. In *International Conference on Computer Design (ICCD)*, pages 542–545, 2001. https://doi.org/10.1109/ICCD.2001.955088.

[FL02] A.M. Fiskiran and R.B. Lee. Workload characterization of elliptic curve cryptography and other network security algorithms for constrained environments. In *IEEE International Workshop on Workload Characterization (WWC)*, pages 127–137, 2002. https://doi.org/10.1109/WWC.2002.1226501.

[FL04] A.M. Fiskiran and R.B. Lee. Evaluating instruction set extensions for fast arithmetic on binary finite fields. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 125–136, 2004. https://doi.org/10.1109/ASAP.2004.1342464.

[FL05a] A.M. Fiskiran and R.B. Lee. Fast parallel table lookups to accelerate symmetric-key cryptography. In *Information Technology: Coding and Computing (ITCC)*, volume 1, pages 526–531, 2005. https://doi.org/10.1109/ITCC.2005.151.

[FL05b] A.M. Fiskiran and R.B. Lee. On-chip lookup tables for fast symmetric-key encryption. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 356–363, 2005. https://doi.org/10.1109/ASAP.2005.49.

[FLO18] A. Faz-Hernandez, J. López, and A.K.D.S. de Oliveira. SoK: A performance evaluation of cryptographic instruction sets on modern architectures. In *ASIA Public-Key Cryptography Workshop (APKC)*, pages 9–18, 2018. https://doi.org/10.1145/3197507.3197511.

[FMU07] N. Fournel, M. Minier, and S. Ubéda. Survey and benchmark of stream ciphers for wireless sensor networks. In *Workshop on Information Security Theory and Practices (WISTP)*, LNCS 4462, pages 202–214. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-72354-7_17.

[Fou07] J.J.A. Fournier. *Vector microprocessors for cryptography*. PhD thesis, University of Cambridge, 2007. https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-701.html.

[FPP08] D. Fronte, A. Perez, and E. Payrat. Celator: A multi-algorithm cryptographic co-processor. In *Reconfigurable Computing and FPGAs (ReConFig)*, pages 438–443, 2008. https://doi.org/10.1109/ReConFig.2008.76.

[Fri22] T. Fritzmann. *Towards Secure Coprocessors and Instruction Set Extensions for Acceleration of Post-Quantum Cryptography*. PhD thesis, Technischen Universität München, 2022. https://mediatum.ub.tum.de/1650057.

[FSS20a] T. Fritzmann, G. Sigl, and J. Sepúlveda. Extending the RISC-V instruction set for hardware acceleration of the post-quantum scheme LAC. In *Design, Automation, and Test in Europe (DATE)*, pages 1420–1425, 2020. https://doi.org/10.23919/DATE48585.2020.9116567.

[FSS20b] T. Fritzmann, G. Sigl, and J. Sepúlveda. RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020(4):239–280, 2020. https://doi.org/10.13154/tches.v2020.i4.239-280.

[GB11] C. Galuzzi and K. Bertels. The instruction-set extension problem: A survey. *ACM Transactions on Reconfigurable Technology and Systems*, 4(2):18:1–18:28, 2011. https://doi.org/10.1145/1968502.1968509.

[GBG+03] C. Grabbe, M. Bednara, J. von zur Gathen, J. Shokrollahi, and J. Teich. A high performance VLIW processor for finite field arithmetic. In *International Symposium on Parallel and Distributed Processing Workshops (IPDPSW)*, pages 396–403, 2003. https://doi.org/10.1109/IPDPS.2003.1213351.

[GBG+11] M. Grand, L. Bossuet, G. Gogniat, B. Le Gal, J.-P. Delahaye, and D. Dallet. A reconfigurable multi-core cryptoprocessor for multi-channel communication systems. In *International Symposium on Parallel and Distributed Processing Workshops (IPDPSW)*, pages 204–211, 2011. https://doi.org/10.1109/IPDPS.2011.143.

[GFG] S. Gueron, W.K. Feghali, and V. Gopal. Architecture and instruction set for implementing advanced encryption standard (AES). U.S. Patent 7949130B2. https://patents.google.com/patent/US7949130B2.

[GGH+11] P. Grabher, J. Großschädl, S. Hoerder, K. Järvinen, D. Page, S. Tillich, and M. Wójcik. An exploration of mechanisms for dynamic cryptographic instruction set extension. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 6917, pages 1–16. Springer-Verlag, 2011. https://doi.org/10.1007/978-3-642-23951-9_1.

[GGM+21] S. Gao, J. Großschädl, B. Marshall, D. Page, T.H. Pham, and F. Regazzoni. An instruction set extension to support software-based masking. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2021(4):283–325, 2021. https://doi.org/10.46586/tches.v2021.i4.283-325.

[GGP08] P. Grabher, J. Großschädl, and D. Page. Light-weight instruction set extensions for bit-sliced cryptography. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 5154, pages 331–345. Springer-Verlag, 2008. https://doi.org/10.1007/978-3-540-85053-3_21.

[GGPY89] P.P. Gelsinger, P.A. Gargini, G.H. Parker, and A.Y.C. Yu. Microprocessors circa 2000. *IEEE Spectrum*, 26(10):43–47, 1989. https://doi.org/10.1109/6.40684.

[GGY+13] S. Gulley, V. Gopal, K. Yap, W. Feghali, J. Guilford, and G. Wolrich. Intel SHA Extensions: New Instructions Supporting the Secure Hash Algorithm on Intel Architecture Processors. Technical report, Intel Corp., 2013. https://www.intel.com/content/dam/develop/external/us/en/documents/intel-sha-extensions-white-paper-402097.pdf.

[GIP+06] J. Großschädl, P. Ienne, L. Pozzi, S. Tillich, and A.K. Verma. Combining algorithm exploration with instruction set design: a case study in elliptic curve cryptography. In *Design, Automation, and Test in Europe (DATE)*, pages 218–223, 2006. https://doi.org/10.1109/DATE.2006.244089.

[GK03a] J. Großschädl and G.-A. Kamendje. Instruction set extension for fast elliptic curve cryptography over binary finite fields GF(2^m). In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 455–468, 2003. https://doi.ieeecomputersociety.org/10.1109/ASAP.2003.1212868.

[GK03b] J. Großschädl and G.-A. Kamendje. Low-power design of a functional unit for arithmetic in finite fields GF(p) and GF(2^m). In *Workshop on Information Security Applications (WISA)*, LNCS 2908, pages 227–243. Springer-Verlag, 2003. https://doi.org/10.1007/978-3-540-24591-9_18.

[GKM+11] P. Gauravaram, L.R. Knudsen, K. Matusiewicz, F. Mendel, C. Rechberger, M. Schläaffer, and S.S. Thomsen. Grøstl – a SHA-3 candidate. Technical report, 2011. http://www.groestl.info/Groestl.pdf.

[GKP04] J. Großschädl, S.S. Kumar, and C. Paar. Architectural support for arithmetic in optimal extension fields. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 111–124, 2004. https://doi.org/10.1109/ASAP.2004.10004.

[GL24] O. Ganon and I. Levi. CrISA-X: Unleashing performance excellence in lightweight symmetric cryptography for extendable and deeply embedded processors. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(3):377–417, 2024. https://doi.org/10.46586/tches.v2024.i3.377-417.

[GLM24] C. Gewehr, L. Luza, and F.G. Moraes. Hardware acceleration of Crystals-Kyber in low-complexity embedded systems with RISC-V instruction set extensions. *IEEE Access*, 12:94477–94495, 2024. https://doi.org/10.1109/ACCESS.2024.3416812.

[GM23] C.G.D.A. Gewehr and F.G. Moraes. Improving the efficiency of cryptography algorithms on resource-constrained embedded systems via RISC-V instruction set extensions. In *SBC/SBMicro/IEEE/ACM Symposium on Integrated Circuits and Systems Design (SBCCI)*, pages 1–6, 2023. https://doi.org/10.1109/SBCCI60457.2023.10261964.

[GMPP20] S. Gao, B. Marshall, D. Page, and T.H. Pham. FENL: an ISE to mitigate analogue micro-architectural leakage. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2020(2):73–98, 2020. https://doi.org/10.13154/tches.v2020.i2.73-98.

[GMT24] C. Gouert, D. Mouris, and N.G. Tsoutsos. Juliet: A configurable processor for computing on encrypted data. *IEEE Transactions on Computers*, 73:2335–2349, 2024. https://doi.org/10.1109/TC.2024.3416752.

[Gon00] R.E. Gonzalez. Xtensa: A configurable and extensible processor. *IEEE Micro*, 20(2):60–70, 2000. https://doi.org/10.1109/40.848473.

[GP12] H. Groß and T. Plos. On using instruction-set extensions for minimizing the hardware-implementation costs of symmetric-key algorithms on a low-resource microcontroller. In *Radio Frequency Identification: Security and Privacy Issues (RFIDSec)*, LNCS 7739, pages 149–164. Springer-Verlag, 2012. https://doi.org/10.1007/978-3-642-36140-1_11.

[GPM23] B. Gigerl, R. Primas, and S. Mangard. Secure context switching of masked software implementations. In *Asia Conference on Computer and Communications Security (Asia CCS)*, pages 980–992, 2023. https://doi.org/10.1145/3579856.3595798.

[GPS08] S.D. Galbraith, K.G. Paterson, and N.P. Smart. Pairings for cryptographers. *Discrete Applied Mathematics*, 156:3113–3121, 2008. https://doi.org/10.1016/j.dam.2007.12.010.

[Gro03] J. Großschädl. Architectural support for long integer modulo arithmetic on RISC-based smart cards. *International Journal of High Performance Computing Applications (IJHPCA)*, 17(2):135–146, 2003. https://doi.org/10.1177/1094342003017002004.

[GS04] J. Großschädl and E. Savaş. Instruction set extensions for fast arithmetic in finite fields $GF(p)$ and $GF(2^m)$. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 3156, pages 133–147. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-28632-5_10.

[Gue09] S. Gueron. Intel's new AES instructions for enhanced performance and security. In *Fast Software Encryption (FSE)*, LNCS 5665, pages 51–66. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-03317-9_4.

[Gue10] S. Gueron. Intel Advanced Encryption Standard (AES) New Instructions Set. Technical Report 323641-001, Intel Corp., 2010. https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf.

[GYH18] Q. Ge, Y. Yarom, and G. Heiser. No security without time protection: we need a new hardware-software contract. In *Asia-Pacific Workshop on Systems (APSys)*, 2018. https://doi.org/10.1145/3265723.3265724.

[HCP+16] M.D. Hill, D. Christie, D. Patterson, J.J. Yi, D. Chiou, and R. Sendag. Proprietary versus open instruction sets. *IEEE Micro*, 36(4):58–68, 2016. https://doi.org/10.1109/MM.2016.61.

[HFHK03] S. Hauck, T.W. Fry, M.M. Hosler, and J.P. Kao. The Chimaera reconfigurable functional unit. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 12(2):206–217, 2003. https://doi-org.bris.idm.oclc.org/10.1109/TVLSI.2003.821545.

[HGTW05] S. Hines, J. Green, G. Tyson, and D. Whalley. Improving program efficiency by packing instructions into registers. In *International Symposium on Computer Architecture (ISCA)*, pages 260–271, 2005. https://doi.org/10.1109/ISCA.2005.32.

[HHB24] J. Haring, V. Hadzic, and R. Bloem. Closing the gap: Leakage contracts for processors with transitions and glitches. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(4):110–132, 2024. https://doi.org/10.46586/tches.v2024.i4.110-132.

[Hil08] Y. Hilewitz. *Advanced Bit Manipulation Instructions: Architecture, Implementation And Applications*. PhD thesis, Princeton University, 2008.

[HKM12] M. Hamburg, P.C. Kocher, and M.E. Marson. Analysis of Intel's Ivy Bridge digital random number generator. Cryptography Research, Inc., 2012. https://www.rambus.com/wp-content/uploads/2015/08/Intel_TRNG_Report_20120312.pdf.

[HV04] A. Hodjat and I. Verbauwhede. Interfacing a high speed crypto accelerator to an embedded CPU. In *Asilomar Conference on Signals, Systems and Computers (ACSSC)*, pages 488–492, 2004. https://doi.org/10.1109/ACSSC.2004.1399180.

[HV11] A. Hakkala and S. Virtanen. Accelerating cryptographic protocols: A review of theory and technologies. In *Communication Theory, Reliability, and Quality of Service (CTRQ)*, pages 103–109, 2011. https://www.utupub.fi/handle/10024/165631.

[HW97] J. Hauser and J. Wawrzynek. Garp: A MIPS processor with a reconfigurable coprocessor. In *Field-Programmable Custom Computing Machines (FCCM)*, pages 12–21, 1997. https://doi.org/10.1109/FPGA.1997.624600.

[HYL08] Y. Hilewitz, Y.L. Yin, and R.B. Lee. Accelerating the Whirlpool hash function using parallel table lookup and fast cyclical permutation. In *Fast Software Encryption (FSE)*, LNCS 5086, pages 173–188. Springer-Verlag, 2008. https://doi.org/10.1007/978-3-540-71039-4_11.

[IL07] P. Ienne and R. Leupers, editors. *Customizable Embedded Processors*. Systems on Silicon. Morgan Kaufmann, 2007. https://doi.org/10.1016/B978-0-12-369526-0.X5000-1.

[Inc20] Microchip Technology Inc. AVR instruction set manual. Technical Report DS40002198A, 2020. https://ww1.microchip.com/downloads/en/devicedoc/AVR-Instruction-Set-Manual-DS40002198A.pdf.

[Int18] Intel. Intel digital random number generator (DRNG), software implementation guide. Revision 2.1, 2018. https://www.intel.com/content/dam/develop/external/us/en/documents/drng-software-implementation-guide-2-1-185467.pdf.

[Int22a] Intel 64 and IA-32 architectures – software developer's manual (volume 1: Basic architecture). Technical Report 253665-077US, Intel Corp., 2022. http://software.intel.com/en-us/articles/intel-sdm.

[Int22b] Intel 64 and IA-32 architectures – software developer's manual (volume 2: Instruction set reference a-z). Technical Report 325383-077US, Intel Corp., 2022. http://software.intel.com/en-us/articles/intel-sdm.

[IPK+24] J.L. Imaña, L. Piñuel, Y.-M. Kuo, O. Ruano, and F. Garcìa-Herrero. Efficient low-latency multiplication architecture for NIST trinomials with RISC-V integration. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 71:3915–3919, 2024. https://doi.org/10.1109/TCSII.2024.3369103.

[ISP24] T.M. Ignatius, T.B. Singha, and R.P. Palathinkal. Power side-channel attacks on crypto-core based on RISC-V ISA for high-security applications. *IEEE Access*, 12:150230–150248, 2024. https://doi.org/10.1109/ACCESS.2024.3477961.

[ITAO20] E.N. Işman, C. Topal, L. Akçay, and B. Örs. Instruction extension of an open source RV32IMC core for NTRU cryptosystem. In *European Conference on Circuit Theory and Design (ECCTD)*, pages 1–5, 2020. https://doi.org/10.1109/ECCTD49232.2020.9218372.

[JBM24] A. Jayasena, R. Bachmann, and P. Mishra. CISELeaks: Information leakage assessment of cryptographic instruction set extension prototypes. Cryptology ePrint Archive, Paper 2024/932, 2024. https://eprint.iacr.org/2024/932.

[JSG10] C. Jenkins, M. Schulte, and J. Glossner. Instruction set extensions for the advanced encryption standard on a multithreaded software defined radio platform. *International Journal of High Performance Systems Architecture (IJHPSA)*, 2(3/4):203–214, 2010. https://doi.org/10.1504/IJHPSA.2010.034541.

[KBB+14] T. Kluter, S. Burri, P. Brisk, E. Charbon, and P. Ienne. Virtual ways: Low-cost coherence for instruction set extensions with architecturally visible storage. *ACM Transactions on Architecture and Code Optimisation*, 11(2):1–26, 2014. https://doi.org/10.1145/2576877.

[KBCI14] T. Kluter, P. Brisk, E. Charbon, and P. Ienne. Way stealing: A unified data cache and architecturally visible storage for instruction set extensions. *IEEE Transactions On Very Large Scale Integration (VLSI) Systems*, 22(1):62–75, 2014. https://doi.org/10.1109/TVLSI.2012.2236689.

[KBH99] B. Kastrup, A. Bink, and J. Hoggerbrugge. ConCISe: A compiler driven CPLD-based instruction set accelerator. In *Field-Programmable Custom Computing Machines (FCCM)*, pages 92–101, 1999. https://doi.org/10.1109/FPGA.1999.803671.

[KBIC09] T. Kluter, P. Brisk, P. Ienne, and E. Charbon. Way stealing: cache-assisted automatic instruction set extensions. In *Design Automation Conference (DAC)*, pages 31–36, 2009. https://doi.org/10.1145/1629911.1629923.

[KCS23] P. Kiaei, T. Conroy, and P. Schaumont. Architecture support for bitslicing. *IEEE Transactions on Emerging Topics in Computing*, 11:497–510, 2023. https://doi.ieeecomputersociety.org/10.1109/TETC.2022.3215480.

[KGM21] Y.-M. Kuo, F. Garcia-Herrero, and J.A. Maestro. Versatile RISC-V ISA Galois field arithmetic extension for cryptography and error-correction codes. In *Computer Architecture Research with RISC-V (CARRV)*, 2021. https://carrv.github.io/2021.

[KGRM23] Y.-M. Kuo, F. García-Herrero, O. Ruano, and J.A. Maestro. RISC-V Galois field ISA extension for non-binary error-correction codes and classical and post-quantum cryptography. *IEEE Transactions on Computers*, 72:682–692, 2023. https://doi.org/10.1109/TC.2022.3174587.

[KH25] S. Kennedy and B. Halak. Acceleration of McEliece cryptosystem with instruction set extension for RISC-V. In *IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 996–1001, 2025. https://doi.org/10.1109/CSR64739.2025.11130090.

[KKPY24] M.J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang. pqm4: Benchmarking NIST additional post-quantum signature schemes on microcontrollers. Cryptology ePrint Archive, Paper 2024/112, 2024. https://eprint.iacr.org/2024/112.

[KL21] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. CRC Press, 3 edition, 2021. https://www.cs.umd.edu/~jkatz/imc.html.

[KLA+19] K. Kiningham, P. Levis, M. Anderson, D. Boneh, M. Horowitz, and M. Shih. Falcon - a flexible architecture for accelerating cryptography. In *Mobile Ad Hoc and Sensor Systems (MASS)*, pages 136–144, 2019. https://doi.org/10.1109/MASS.2019.00025.

[KLS+23] M. Krausz, G. Land, F. Stolz, D. Naujoks, J. Richter-Brockmann, T. Güneysu, and L. Kogelheide. To extend or not to extend: Agile masking instructions for PQC. Cryptology ePrint Archive, Paper 2023/1287, 2023. https://eprint.iacr.org/2023/1287.

[KMD+20] P. Kiaei, D. Mercadier, P.-E. Dagand, K. Heydemann, and P. Schaumont. Custom instruction support for modular defense against side-channel and fault attacks. In *Constructive Side-Channel Analysis and Secure Design (COSADE)*, LNCS 12244, pages 221–253. Springer-Verlag, 2020. https://doi.org/10.1007/978-3-030-68773-1_11.

[KMPZ95] L. Kohn, G. Maturana, A. Prabhu, and G. Zyner. The visual instruction set (VIS) in UltraSPARC. In *Technologies for the Information Superhighway (COMPCON)*, pages 462–469, 1995. https://doi.org/10.1109/CMPCON.1995.512423.

[KP04] S.S. Kumar and C. Paar. Reconfigurable instruction set extension for enabling ECC on an 8-bit processor. In *Field Programmable Logic and Application (FPL)*, LNCS 3203, pages 586–595. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-30117-2_60.

[KR07] I. Kuon and J. Rose. Measuring the gap between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 26(2):203–215, 2007. https://doi.org/10.1109/TCAD.2006.884574.

[KR10] I. Kuon and J. Rose. *Quantifying and Exploring the Gap Between FPGAs and ASICs*. Springer, 2010. https://doi.org/10.1007/978-1-4419-0739-4.

[KS20] P. Kiaei and P. Schaumont. Domain-oriented masked instruction set architecture for RISC-V. Cryptology ePrint Archive, Report 2020/465, 2020. https://eprint.iacr.org/2020/465.

[KSG08] Ö. Kocabas, E. Savaş, and J. Großschädl. Enhancing an embedded processor core with a cryptographic unit for speed and security. In *Reconfigurable Computing and FPGAs (ReConFig)*, pages 409–414, 2008. https://doi.org/10.1109/ReConFig.2008.59.

[KSMGS24] P. Van Kempen, M. Salmen, D. Mueller-Gritschneder, and U. Schlichtmann. Seal5: Semi-automated LLVM support for RISC-V ISA extensions including autovectorization. In *Euromicro Conference on Digital System Design (DSD)*, pages 335–342, 2024. https://doi.org/10.1109/DSD64264.2024.00052.

[KZS+09] D. Kammler, D. Zhang, P. Schwabe, H. Scharwaechter, M. Langenberg, D. Auras, G. Ascheid, and R. Mathar. Designing an ASIP for cryptographic pairings over Barreto-Naehrig curves. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 5747, pages 254–271. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-04138-9_19.

[LC08] S.H. Lee and L. Choi. Accelerating symmetric and asymmetric ciphers with register file extension for multi-word and long-word operation. In *International Conference on Information Science and Security (ICISS)*, pages 102–107, 2008. https://doi.org/10.1109/ICISS.2008.32.

[LC10] R.B. Lee and Y.-Y. Chen. Processor accelerator for AES. In *Symposium on Application Specific Processors (SASP)*, pages 16–21, 2010. https://doi.org/10.1109/SASP.2010.5521153.

[LCW18] Y. Liu, R.C.C. Cheung, and H. Wong. Lightweight secure processor prototype on FPGA. In *Field Programmable Logic and Applications (FPL)*, pages 443–444, 2018. https://doi.org/10.1109/FPL.2018.00081.

[LDH06] Y.W. Law, J. Doumen, and P. Hartel. Survey and benchmark of block ciphers for wireless sensor networks. *ACM Transactions on Sensor Networks (TOSN)*, 2(1):65–93, 2006. http://doi.org/10.1145/1138127.1138130.

[Lee95] R.B. Lee. Accelerating multimedia with enhanced micro-processors. *IEEE Micro*, 15(2):22–32, 1995. https://doi.org/10.1109/40.372347.

[Lee96] R.B. Lee. Subword parallelism with MAX-2. *IEEE Micro*, 16(4):51–59, 1996. https://doi.org/10.1109/40.526925.

[Lee03] R.B. Lee. Challenges in the design of security-aware processors. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 1–2, 2003. https://doi.org/10.1109/ASAP.2003.1212824.

[LF05] R.B. Lee and A.M. Fiskiran. PLX: An instruction set architecture and testbed for multimedia information processing. *VLSI Signal Processing Systems for Signal, Image and Video Technology*, 40(1):85–108, 2005. https://doi.org/10.1007/s11265-005-4940-8.

[LH96] R.B. Lee and J. Huck. 64-bit and multimedia extensions in the PA-RISC 2.0 architecture. In *Technologies for the Information Superhighway (COMPCON)*, pages 152–160, 1996. https://doi.org/10.1109/CMPCON.1996.501762.

[LHP20] T. Li, B. Hopkins, and S. Parameswaran. SIMF: Single-instruction multiple-flush mechanism for processor temporal isolation. *cs.CR*, 2011.10249, 2020. https://arxiv.org/abs/2011.10249.

[LM90] X. Lai and J.L. Massey. A proposal for a new block encryption standard. In *Advances in Cryptology (EUROCRYPT)*, LNCS 473, pages 389–404. Springer-Verlag, 1990. https://doi.org/10.1007/3-540-46877-3_35.

[LMP22] H. Li, N. Mentens, and S. Picek. A scalable SIMD RISC-V based processor with customized vector extensions for CRYSTALS-Kyber. In *Design Automation Conference (DAC)*, pages 733–738, 2022. https://doi.org/10.1145/3489517.3530552.

[LMP23] H. Li, N. Mentens, and S. Picek. Maximizing the potential of custom RISC-V vector extensions for speeding up SHA-3 hash functions. In *Design, Automation, and Test in Europe (DATE)*, pages 1–6. IEEE, 2023. https://doi.org/10.23919/DATE56975.2023.10137009.

[LQYW24] L. Li, G. Qin, Y. Yu, and W. Wang. Compact instruction set extensions for Kyber. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 43(3):756–760, 2024. https://doi.org/10.1109/TCAD.2023.3327104.

[LT23] F. Lozachmeur and A. Tisserand. A RISC-V instruction set extension for flexible hardware/software protection of cryptosystems masked at high orders. In *IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 360–364, 2023. https://doi.org/10.1109/MWSCAS57524.2023.10405991.

[LTQ+24] L. Li, Q. Tian, G. Qin, S. Chen, and W. Wang. Compact instruction set extensions for Dilithium. *ACM Transactions on Embedded Computing Systems*, 23(2):23:1–23:21, 2024. https://doi.org/10.1145/3643826.

[LTZ+25] Y. Lan, L. Tang, N. Zhang, Y. Eum, J. Hoe, and F. Franchetti. A RISC-V vector extension for multi-word arithmetic. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1684–1693, 2025. https://doi.org/10.1145/3731599.3767530.

[Lu21] T. Lu. A survey on RISC-V security: Hardware and architecture. *cs.CR*, 2107.04175, 2021. https://arxiv.org/abs/2107.04175.

[LYS04] R.B. Lee, X. Yang, and Z. Shi. Validating word-oriented processors for bit and multi-word operations. In *Advances in Computer Systems Architecture (ACSAC)*, LNCS 3189, pages 473–488. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-30102-8_40.

[MA07] D. Montgomery and A. Akoglu. Methodology and toolset for ASIP design and development targeting cryptography-based applications. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 365–370, 2007. https://doi.org/10.1109/ASAP.2007.4459291.

[Man11] R. Manley. *Program generation for Intel AES new instructions*. PhD thesis, Trinity College Dublin, 2011.

[Mar19] B. Marshall. On hardware verification in an open source context. In *Workshop on Open Source Design Automation (OSDA)*, 2019. https://osda.gitlab.io/19/1.2.pdf.

[MBB+23] K. Miteloudi, J. Bos, O. Bronchain, B. Fay, and J. Renes. PQ.V.ALU.E: Post-quantum RISC-V custom ALU extensions on Dilithium and Kyber. In *Smart Card Research and Advanced Applications (CARDIS)*, LNCS 14530, pages 190–209. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-54409-5_10.

[MCR18] N. Mentens, E. Charbon, and F. Regazzoni. Rethinking secure FPGAs: Towards a cryptography-friendly configurable cell architecture and its automated design flow. In *Field-Programmable Custom Computing Machines (FCCM)*, pages 215–215, 2018. https://doi.org/10.1109/FCCM.2018.00049.

[MD07] S. Majzoub and H. Diab. Instruction-set extension for cryptographic applications on reconfigurable platform. *Journal of Circuits, Systems and Computers*, 16(6):911–927, 2007. https://doi.org/10.1142/S0218126607004076.

[MG10] R. Manley and D. Gregg. A program generator for Intel AES-NI instructions. In *Progress in Cryptology (INDOCRYPT)*, LNCS 6498, pages 311–327. Springer-Verlag, 2010. https://doi.org/10.1007/978-3-642-17401-8_22.

[MGR24] G. Mohr, M. Guarnieri, and J. Reineke. Synthesizing hardware-software leakage contracts for RISC-V open-source processors. In *Design, Automation, and Test in Europe (DATE)*, pages 1–6, 2024. https://doi.org/10.23919/DATE58400.2024.10546681.

[MIPS01b] MIPS32 architecture for programmers, volume II: The MIPS32 instruction set. Technical report, MIPS Technologies, Inc., 2001.

[MMG10] R. Manley, P. Magrath, and D. Gregg. Code generation for hardware accelerated AES. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 345–348, 2010. https://doi.org/10.1109/ASAP.2010.5540955.

[MMW21] C. Meijer, V. Moonsamy, and J. Wetzels. Where's crypto?: Automated identification and classification of proprietary cryptographic primitives in binary code. In *USENIX Security Symposium*, pages 555–572, 2021. https://www.usenix.org/conference/usenixsecurity21/presentation/meijer.

[MNP+21] B. Marshall, G.R. Newell, D. Page, M.-J.O. Saarinen, and C. Wolf. The design of scalar AES instruction set extensions for RISC-V. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2021(1):109–136, 2021. https://doi.org/10.46586/tches.v2021.i1.109-136.

[MP21] B. Marshall and D. Page. SME: Scalable Masking Extensions. Cryptology ePrint Archive, Paper 2021/1416, 2021. https://eprint.iacr.org/2021/1416.

[MPP20] B. Marshall, D. Page, and T.H. Pham. Implementing the draft RISC-V scalar cryptography extensions. In *Hardware and Architectural Support for Security and Privacy (HASP)*, 2020. https://doi.org/10.1145/3458903.3458904.

[MPP21] B. Marshall, D. Page, and T. Pham. A lightweight ISE for ChaCha on RISC-V. In *Application-specific Systems, Architectures and Processors (ASAP)*, pages 25–32. IEEE, 2021. https://doi.org/10.1109/ASAP52443.2021.00011.

[MTR+99] E. Mosanya, C. Teuscher, H. Restrepo, P. Galley, and E. Sanchez. CryptoBooster: A reconfigurable and modular cryptographic coprocessor. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 1717, pages 246–256. Springer-Verlag, 1999. https://doi.org/10.1007/3-540-48059-5_21.

[NDZ+21] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci. A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms. *IEEE Access*, 9:150798–150808, 2021. https://doi.org/10.1109/ACCESS.2021.3126208.

[NIK04] K. Nadehara, M. Ikekawa, and I. Kuroda. Extended instructions for the AES cryptography and their efficient implementation. In *Signal Processing Systems (SIPS)*, pages 152–157, 2004. https://doi.org/10.1109/SIPS.2004.1363041.

[NIS16] NIST cryptographic standards and guidelines development process. National Institute of Standards and Technology (NIST) Internal Report 7977, 2016. https://doi.org/10.6028/NIST.IR.7977.

[Nis21] G. Nishandji. Performance evaluation of cryptographic algorithms in software and with hardware implementation of specialized instructions for the RISC-V instruction set architecture. MSc thesis, Northern Arizona University, 2021. https://openknowledge.nau.edu/id/eprint/5648.

[NIST01] Advanced Encryption Standard (AES). National Institute of Standards and Technology (NIST) Federal Information Processing Standard (FIPS) 197, 2001. http://csrc.nist.gov.

[NIST07] Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. National Institute of Standards and Technology (NIST) Special Publication 800-38D, 2007. http://csrc.nist.gov.

[NIST15] Secure Hash Standard (SHS). National Institute of Standards and Technology (NIST) Federal Information Processing Standard (FIPS) 180-4, 2015. http://csrc.nist.gov.

[NIST99] Data Encryption Standard (DES). National Institute of Standards and Technology (NIST) Federal Information Processing Standard (FIPS) 46-3, 1999. http://csrc.nist.gov.

[NOOS95] E. Nahum, S. O'Malley, H. Orman, and R. Schroeppel. Towards high performance cryptographic software. In *High Performance Communication Subsystems (HPCS)*, pages 69–72, 1995. https://doi.org/10.1109/HPCS.1995.662009.

[NPL+25] V.T. Nguyen, P.H. Pham, V.T.D. Le, H.L. Pham, T.H. Vu, and T.D. Tran. AES-RV: Hardware-efficient RISC-V accelerator with low-latency AES instruction extension for IoT security. *IEICE Electronics Express*, page 22.20250329, 2025. https://doi.org/10.1587/elex.22.20250329.

[NRE+12] K.D. Nima, E.A. Reza, A. Erfan, T. Ahmad, R. Mahsa, and M. Mina. CIARP: Crypto instruction-aware RISC processor. In *IEEE Symposium on Computers & Informatics (ISCI)*, pages 208–213, 2012. https://doi.org/10.1109/ISCI.2012.6222696.

[NST10] A. Nohl, F. Schirrmeister, and D. Taussig. Application specific processor design: Architectures, design methods and tools. In *International Conference on Computer-Aided Design (ICCAD)*, pages 349–352, 2010. https://doi.org/10.1109/ICCAD.2010.5653632.

[ODKM+24] J. Oppermann, B.M. Damian-Kosterhon, F. Meisel, T. Mürmann, E. Jentzsch, and A. Koch. Longnail: High-level synthesis of portable custom instruction set extensions for RISC-V processors from descriptions in the open-source CoreDSL language. In *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 591–606, 2024. https://doi.org/10.1145/3620666.3651375.

[OFP+24] F. Oberhansl, T. Fritzmann, T. Pöppelmann, D.B. Roy, and G. Sigl. Uniform instruction set extensions for multiplications in contemporary and post-quantum cryptography. *Journal of Cryptographic Engineering (JCEN)*, 14(1):1–18, 2024. https://doi.org/10.1007/s13389-023-00332-2.

[OFW99] S. Oberman, G. Favor, and F. Weber. AMD 3DNow! technology: architecture and implementations. *IEEE Micro*, 19(2):37–48, 1999. https://doi.org/10.1109/40.755466.

[OGM+13] E. Owusu, J. Guajardo, J. McCune, J. Newsome, A. Perrig, and A. Vasudevan. OASIS: on achieving a sanctuary for integrity and secrecy on untrusted platforms. In *Computer and Communications Security (CCS)*, pages 13–24, 2013. https://doi.org/10.1145/2508859.2516678.

[Paa02] C. Paar. The future of the art of cryptographic implementations. In *STORK Cryptography Workshop*, 2002. http://www.stork.eu.org/program.html.

[PAI06] L. Pozzi, K. Atasu, and P. Ienne. Exact and approximate algorithms for the extension of embedded processor instruction sets. *IEEE Transactions on Computers*, 25(7):1209–1229, 2006. https://doi.org/10.1109/TCAD.2005.855950.

[PBI+10] N. Pothineni, P. Brisk, P. Ienne, A. Kumar, and K. Paul. A high-level synthesis flow for custom instruction set extensions for application-specific processors. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 707–712, 2010. https://doi.org/10.1109/ASPDAC.2010.5419795.

[PD80] D.A. Patterson and D.R. Ditzel. The case for the reduced instruction set computer. *SIGARCH Comput. Archit. News*, 8(6):25–33, 1980. https://doi.org/10.1145/641914.641917.

[PDM+25] V. Piscopo, A. Dolmeta, M. Mirigaldi, M. Martina, and G. Masera. A deep dive into integration methodologies in RISC-V. In *ACM International Conference on Computing Frontiers (CF): Workshops and Special Sessions*, pages 30–33, 2025. https://doi.org/10.1145/3706594.3726969.

[Pet25] R. Petri. *Lightweight Software-Hardware Codesign for Post-Quantum Cryptography*. PhD thesis, Syddansk Universitet, 2025. https://doi.org/10.21996/6a13dbbc-9e4f-4577-a934-fdae2a5471c6.

[PI05] L. Pozzi and P. Ienne. Exploiting pipelining to relax register-file port constraints of instruction-set extensions. In *Compilers, Architectures, and Synthesis for Embedded Systems (CASES)*, pages 2–10, 2005. https://doi.org/10.1145/1086297.1086300.

[Pow15] IBM. Power ISA. Technical Report 3.0, 2015. https://openpowerfoundation.org/specifications/isa.

[Pow18] Power ISA. Technical Report 2.07 B, IBM, 2018. https://ibm.ent.box.com/s/jd5w15gz301s5b5dt375mshpq9c3lh4u.

[PS81] D.A. Patterson and C.H. Séquin. RISC I: A reduced instruction set VLSI computer. In *International Symposium on Computer Architecture (ISCA)*, pages 216–230, 1981. https://doi.org/10.1145/285930.285981.

[PS82] D.A. Patterson and C.H. Séquin. A VLSI RISC. *IEEE Computer*, 15(9):8–21, 1982. https://doi.org/10.1109/MC.1982.1654133.

[PSP08] C. Puttmann, J. Shokrollahi, and M. Porrmann. Resource efficiency of instruction set extensions for elliptic curve cryptography. In *Information Technology New Generations (ITNG)*, pages 131–136, 2008. https://doi.org/10.1109/ITNG.2008.130.

[PV17] K. Papagiannopoulos and N. Veshchikov. Mind the gap: Towards secure 1st-order masking in software. In *Constructive Side-Channel Analysis and Secure Design (COSADE)*, LNCS 10348, pages 282–297. Springer-Verlag, 2017. https://doi.org/10.1007/978-3-319-64647-3_17.

[PVI02] L. Pozzi, M. Vuletic, and P. Ienne. Automatic topology-based identification of instruction-set extensions for embedded processors. In *Design, Automation, and Test in Europe (DATE)*, pages 1138–1138, 2002. https://doi.org/10.1109/DATE.2002.998496.

[PW96] A. Peleg and U. Weiser. MMX technology extension to the Intel architecture. *IEEE Micro*, 16(4):42–50, 1996. https://doi.org/10.1109/40.526924.

[Raw16] H.K. Rawat. Vector instruction set extensions for efficient and reliable computation of Keccak. MSc thesis, Virginia Polytechnic Institute and State University, 2016. https://vtechworks.lib.vt.edu/bitstream/handle/10919/72857/Rawat_HK_T_2016.pdf.

[RCS+09] F. Regazzoni, A. Cevrero, F.-X. Standaert, S. Badel, T. Kluter, P. Brisk, Y. Leblebici, and P. Ienne. A design flow and evaluation framework for DPA-resistant instruction set extensions. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 5747, pages 205–219. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-04138-9_15.

[Rei19] A. Reid. *Defining interfaces between hardware and software: Quality and performance*. PhD thesis, School of Computing Science, University of Glasgow, 2019. http://theses.gla.ac.uk/41068.

[REP+09] F. Regazzoni, T. Eisenbarth, A. Poschmann, J. Großschädl, F. Gurkaynak, M. Macchetti, Z. Toprak, L. Pozzi, C. Paar, Y. Leblebici, and P. Ienne. Evaluating resistance of MCML technology to power analysis attacks using a simulation-based methodology. In *Transactions on Computational Science IV*, LNCS 5430, pages 230–243. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-01004-0_13.

[Res18] E. Rescorla. The Transport Layer Security (TLS) protocol version 1.3. Internet Engineering Task Force (IETF) Request for Comments (RFC) 8446, 2018. http://tools.ietf.org/html/rfc8446.

[RI16] F. Regazzoni and P. Ienne. Instruction set extensions for secure applications. In *Design, Automation, and Test in Europe (DATE)*, pages 1529–1534, 2016.

[RIS22] RISC-V. RISC-V ABIs specification. Technical Report 1.0, RISC-V International, 2022. https://github.com/riscv-non-isa/riscv-elf-psabi-doc/releases/download/v1.0/riscv-abi.pdf.

[RIS24] RISC-V. The RISC-V instruction set manual volume I: Unprivileged architecture. Technical Report 20240411, RISC-V International, 2024. https://github.com/riscv/riscv-isa-manual/releases/download/20240411/unpriv-isa-asciidoc.pdf.

[Riv94] R.L. Rivest. The RC5 encryption algorithm. In *Fast Software Encryption (FSE)*, LNCS 1008, pages 86–96. Springer-Verlag, 1994. https://doi.org/10.1007/3-540-60590-8_7.

[RKL+04] S. Ravi, P.C. Kocher, R.B. Lee, G. McGraw, and A. Raghunathan. Security as a new dimension in embedded system design. In *Design Automation Conference (DAC)*, pages 753–760, 2004. https://doi.org/10.1145/996566.996771.

[RRKH04] S. Ravi, A. Raghunathan, P.C. Kocher, and S. Hattangady. Security in embedded systems: Design challenges. *ACM Transactions on Embedded Computer Systems*, 3(3):461–491, 2004. https://doi.org/10.1145/1015047.1015049.

[RRSY98] R.L. Rivest, M.J.B. Robshaw, R. Sidney, and Y.L. Yin. The RC6 block cipher, 1998. http://people.csail.mit.edu/rivest/pubs/RRSY98.pdf.

[RS16] H.K. Rawat and P. Schaumont. SIMD instruction set extensions for Keccak with applications to SHA-3, Keyak and Ketje. In *Hardware and Architectural Support for Security and Privacy (HASP)*, pages 1–8, 2016. https://doi.org/10.1145/2948618.2948622.

[SA10] M.I. Soliman and G. Y. Abozaid. FastCrypto: parallel AES pipeline extension for general-purpose processors. *Neural, Parallel, and Scientific Computations*, 18(1):47–58, 2010.

[Saa] M.-J.O. Saarinen. sm4ni. https://github.com/mjosaarinen/sm4ni.

[Saa19] M.-J.O. Saarinen. SNEIK on microcontrollers: AVR, ARMv7-M, and RISC-V with custom instructions. Cryptology ePrint Archive, Report 2019/936, 2019. http://eprint.iacr.org/2019/936.

[Saa20] M.-J.O. Saarinen. A lightweight ISA extension for AES and SM4. *CoRR*, abs/2002.07041, 2020. https://arxiv.org/abs/2002.07041.

[Saa25] M.-J.O. Saarinen. Brief comments on Rijndael-256 and the standard RISC-v cryptography extensions. Cryptology ePrint Archive, Paper 2025/1198, 2025. https://eprint.iacr.org/2025/1198.

[SC14] G. Sayilar and D. Chiou. Cryptoraptor: high throughput reconfigurable cryptographic processor. In *International Conference on Computer-Aided Design (ICCAD)*, pages 155–161, 2014. https://doi.org/10.5555/2691365.2691398.

[SCA18] L. Simon, D. Chisnall, and R. Anderson. What you get is what you C: Controlling side effects in mainstream C compilers. In *IEEE European Symposium on Security & Privacy (Euro-S&P)*, pages 1–15, 2018. https://doi.org/10.1109/EuroSP.2018.00009.

[Sco07] M. Scott. Optimal irreducible polynomials for GF(2^m) arithmetic. Cryptology ePrint Archive, Report 2007/192, 2007. https://eprint.iacr.org/2007/192.

[SCS+09] N. Suarez, G.M. Callico, R. Sarmiento, O. Santana, and A.A. Abbo. Processor customization for software implementation of the AES algorithm for wireless sensor networks. In *Power and Timing Modeling, Optimization and Simulation (PATMOS)*, LNCS 5953, pages 326–335. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-11802-9_37.

[SEM17] K. Shahbazi, M. Eshghi, and R.F. Mirzaee. Design and implementation of an ASIP-based cryptography processor for AES, IDEA, and MD5. *Engineering Science and Technology, an International Journal (JESTECH)*, 20(4):1308–1317, 2017. https://doi.org/10.1016/j.jestch.2017.07.002.

[SFSG23] F. Stolz, M. Fyrbiak, P. Sasdrich, and T. Güneysu. Recommendation for a holistic secure embedded ISA extension. In *Applied Cryptography and Network Security (ACNS)*, LNCS 13906, pages 62–84. Springer-Verlag, 2023. https://doi.org/10.1007/978-3-031-33491-7_3.

[Shi04] Z. Shi. *Bit Permutation Instructions: Architecture, Implementation, And Cryptographic Properties*. PhD thesis, Princeton University, 2004.

[SL00] Z. Shi and R.B. Lee. Bit permutation instructions for accelerating software cryptography. In *Application-Specific Systems, Architectures, and Processors (ASAP)*, pages 138–148, 2000. https://doi.org/10.1109/ASAP.2000.862385.

[SLL+00] H. Singh, M. Lee, G. Lu, F.J. Kurdahi, N. Bagherzadeh, and E.M. Chaves Filho. MorphoSys: An integrated reconfigurable system for data-parallel and computation-intensive applications. *IEEE Transactions on Computers*, 49(5):465–481, 2000. https://doi.org/10.1109/12.859540.

[SLP+24] M. Schneider, D. Lain, I. Puddu, N. Dutly, and S. Capkun. Breaking bad: How compilers break constant-time implementations. *cs.CR*, 2410.13489, 2024. https://arxiv.org/abs/2410.13489.

[SNM22] M.-J.O. Saarinen, G.R. Newell, and B. Marshall. Development of the RISC-V entropy source interface. *Journal of Cryptographic Engineering (JCEN)*, 12(4):371–386, 2022. https://doi.org/10.1007/s13389-021-00275-6.

[Sou22] P. Sousa. HACC-V: hardware-assisted cryptographic coprocessor for RISC-V. MSc thesis, Universidade do Minho, 2022. https://repositorium.sdum.uminho.pt/handle/1822/84681.

[SP18] J. Stecklina and T. Prescher. LazyFP: Leaking FPU register state using microarchitectural side-channels. *cs.CR*, 1806.07480, 2018. http://arxiv.org/abs/1806.07480.

[SP21] S. Steinegger and R. Primas. A fast and compact RISC-V accelerator for Ascon and friends. In *Smart Card Research and Advanced Applications (CARDIS)*, LNCS 12609, pages 53–67. Springer-Verlag, 2021. https://doi.org/10.1007/978-3-030-68487-7_4.

[SPA16] Oracle SPARC architecture 2011. Technical Report D1.0.0, Oracle Corp., 2016. https://www.oracle.com/technetwork/server-storage/sun-sparc-enterprise/documentation/140521-ua2011-d096-p-ext-2306580.pdf.

[SPJR22] S. Singh, A. Perais, A. Jimborean, and A. Ros. Exploring instruction fusion opportunities in general purpose processors. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 199–212, 2022. https://doi.org/10.1109/MICRO56248.2022.00026.

[SRH16] S. Saab, P. Rohatgi, and C. Hampel. Side-channel protections for cryptographic instruction set extensions. Cryptology ePrint Archive, Report 2016/700, 2016. https://eprint.iacr.org/2016/700.

[SRRJ04] F. Sun, S. Ravi, A. Raghunathan, and N.K. Jha. Custom-instruction synthesis for extensible-processor platforms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 23(2):216–228, 2004. https://doi.org/10.1109/TCAD.2003.822133.

[SS05] N.T. Slingerland and A.J. Smith. Multimedia extensions for general purpose microprocessors: a survey. *Microprocessors and Microsystems*, 29(5):225–246, 2005. https://doi.org/10.1016/j.micpro.2004.10.002.

[Sto19] K. Stoffelen. Efficient cryptography on the RISC-V architecture. In *Progress in Cryptology (LATINCRYPT)*, LNCS 11774, pages 323–340. Springer-Verlag, 2019. https://doi.org/10.1007/978-3-030-30530-7_16.

[STRG24] F. Schwarz, J.P. Thoma, C. Rossow, and T. Güneysu. KeyVisor – a lightweight ISA extension for protected key handles with CPU-enforced usage policies. *cs.CR*, 2410.01777, 2024. https://arxiv.org/abs/2410.01777.

[TBW+21] J. Trautmann, A. Beckers, L. Wouters, S. Wildermann, I. Verbauwhede, and J. Teich. Semi-automatic locating of cryptographic operations in side-channel traces. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2022(1):345–366, 2021. https://doi.org/10.46586/tches.v2022.i1.345-366.

[TCG+25] Q. Tian, H. Cheng, C. Guo, D. Page, M. Wang, and W. Wang. A code-based ISE to protect Boolean masking in software. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2025(2):293–332, 2025. https://doi.org/10.46586/tches.v2025.i2.293-332.

[TG99] R.R. Taylor and S.C. Goldstein. A high-performance flexible architecture for cryptography. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 1717, pages 231–245. Springer-Verlag, 1999. https://doi.org/10.1007/3-540-48059-5_20.

[TG04] S. Tillich and J. Großschädl. A simple architectural enhancement for fast and flexible elliptic curve cryptography over binary finite fields GF(2^m). In *Advances in Computer Systems Architecture (ACSAC)*, LNCS 3189, pages 282–295. Springer-Verlag, 2004. https://doi.org/10.1007/978-3-540-30102-8_24.

[TG05] S. Tillich and J. Großschädl. Accelerating AES using instruction set extensions for elliptic curve cryptography. In *International Conference Computational Science and Its Applications (ICCSA)*, volume 2 of *LNCS 3481*, pages 665–675. Springer-Verlag, 2005. https://doi.org/10.1007/11424826_70.

[TG06] S. Tillich and J. Großschädl. Instruction set extensions for efficient AES implementation on 32-bit processors. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 4249, pages 270–284. Springer-Verlag, 2006. https://doi.org/10.1007/11894063_22.

[TG07a] S. Tillich and J. Großschädl. Power analysis resistant AES implementation with instruction set extensions. In *Cryptographic Hardware and Embedded Systems (CHES)*, LNCS 4727, pages 303–319. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-74735-2_21.

[TG07b] S. Tillich and J. Großschädl. VLSI implementation of a functional unit to accelerate ECC and AES on 32-bit processors. In *Workshop on the Arithmetic of Finite Fields (WAIFI)*, LNCS 4547, pages 40–54. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-73074-3_5.

[TGS05] S. Tillich, J. Großschädl, and A. Szekely. An instruction set extension for fast and memory-efficient AES implementation. In *Communications and Multimedia Security (CMS)*, LNCS 3677, pages 11–21. Springer-Verlag, 2005. https://doi.org/10.1007/11552055_2.

[TGS+19] E. Tehrani, T. Graba, A. Si-Merabet, S. Guilley, and J.-L. Danger. Classification of lightweight block ciphers for specific processor accelerated implementations. In *International Conference on Electronics, Circuits and Systems (ICECS)*, pages 747–750, 2019. https://doi.org/10.1109/ICECS46596.2019.8965156.

[TGSD20] E. Tehrani, T. Graba, A. Si-Merabet, and J.-L. Danger. RISC-V extension for lightweight cryptography. In *Euromicro Conference on Digital System Design (DSD)*, pages 222–228, 2020. https://doi.org/10.1109/DSD51259.2020.00045.

[TH99] S. Thakkar and T. Huff. Internet streaming SIMD extensions. *IEEE Computer*, 32(12):26–34, 1999. https://doi.org/10.1109/2.809248.

[THM07] S. Tillich, C. Herbst, and S. Mangard. Protecting AES software implementations on 32-bit processors against power analysis. In *Applied Cryptography and Network Security (ACNS)*, LNCS 4521, pages 141–157. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-72738-5_10.

[Til08] S. Tillich. *Instruction Set Extensions for Support of Cryptography on Embedded Systems*. PhD thesis, 2008. https://graz.elsevierpure.com/en/publications/instruction-set-extensions-for-support-of-cryptography-on-embedde.

[TKS10] S. Tillich, M. Kirschbaum, and A. Szekely. SCA-resistant embedded processors: The next generation. In *Annual Computer Security Applications Conference (ACSAC)*, pages 211–220, 2010. https://doi.org/10.1145/1920261.1920293.

[TMC+23] M.S. Turan, K. McKay, D. Chang, L.E. Bassham, J. Kang, N.D. Waller, J.M. Kelsey, and D. Hong. Status report on the final round of the NIST lightweight cryptography standardization process. Technical report, 2023. https://doi.org/10.6028/NIST.IR.8454.

[TSP09] D. Theodoropoulos, A. Siskos, and D. Pnevmatikatos. CCproc: A custom VLIW cryptography co-processor for symmetric-key ciphers. In *Applied Reconfigurable Computing: Architectures, Tools and Applications (ARC)*, pages 318–323. Springer-Verlag, 2009. https://doi.org/10.1007/978-3-642-00641-8_35.

[UK24] H. Uzuner and E.B. Kavun. NLU-V: A family of instruction set extensions for efficient symmetric cryptography on RISC-V. *Cryptography*, 8(1), 2024. https://doi.org/10.3390/cryptography8010009.

[US24] E. Urquhart and F. Stajano. Acceleration of core post-quantum cryptography primitive on open-source silicon platform through hardware/software co-design. In *Cryptology and Network Security (CANS)*, LNCS 14905, pages 144–161. Springer-Verlag, 2024. https://doi.org/10.1007/978-981-97-8013-6_7.

[USM24] F. Uhle, F. Stolz, and A. Moradi. Another evidence to not employ customized masked hardware: Identifying and fixing flaws in SCARV. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(4):133–155, 2024. https://doi.org/10.46586/tches.v2024.i4.133-155.

[VBI08] A.K. Verma, P. Brisk, and P. Ienne. Fast, quasi-optimal, and pipelined instruction-set extensions. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 435–446, 2008. https://doi.org/10.1109/ASPDAC.2008.4483970.

[VCS22] C. Verhamme, G. Cassiers, and F.-X. Standaert. Analyzing the leakage resistance of the NIST's lightweight crypto competition's finalists. In *Smart Card Research and Advanced Applications (CARDIS)*, LNCS 13820, pages 290–308. Springer-Verlag, 2022. https://doi.org/10.1007/978-3-031-25319-5_15.

[VN07] N. Vassiliadis and G. Theodoridis S. Nikolaidis. The ARISE reconfigurable instruction set extensions framework. In *Embedded Computer Systems: Architectures, Modeling and Simulation (SAMOS)*, pages 153–160, 2007. https://doi.org/10.1109/ICSAMOS.2007.4285746.

[VPG07] T. Vejda, D. Page, and J. Großschädl. Instruction set extensions for pairing-based cryptography. In *Pairing-Based Cryptography (Pairing)*, LNCS 4575, pages 208–224. Springer-Verlag, 2007. https://doi.org/10.1007/978-3-540-73489-5_11.

[VSI+19] A. Varici, G. Saglam, S. Ipek, A. Yildiz, S. Gören, A. Aysu, D. Iskender, T.B. Aktemur, and H.F. Ugurdag. Fast and efficient implementation of lightweight crypto algorithm PRESENT on FPGA through processor instruction set extension. In *East-West Design & Test Symposium (EWDTS)*, pages 1–5, 2019. https://doi.org/10.1109/EWDTS.2019.8884397.

[Wat16] A. Waterman. *Design of the RISC-V Instruction Set Architecture*. PhD thesis, University of California at Berkeley, 2016. https://people.eecs.berkeley.edu/~krste/papers/EECS-2016-1.pdf.

[WCLW24] T. Wang, S. Chen, L. Li, and W. Wang. Fast fourier transform and gaussian sampling instructions designed for FALCON digital signature. In *Information Security and Cryptology (INSCRYPT)*, LNCS 15544, pages 444–463. Springer-Verlag, 2024. https://doi.org/10.1007/978-981-96-4734-7_22.

[WMvG+23] Z. Wang, G. Mohr, K. von Gleissenthall, J. Reineke, and M. Guarnieri. Specification and verification of side-channel security for open-source processors via leakage contracts. In *Computer and Communications Security (CCS)*, pages 2128–2142, 2023. https://doi.org/10.1145/3576915.3623192.

[WSG+20] N. Wistoff, M. Schneider, F.K. Gürkaynak, L. Benini, and G. Heiser. Prevention of microarchitectural covert channels on an open-source 64-bit RISC-V core. In *Computer Architecture Research with RISC-V (CARRV)*, 2020. https://carrv.github.io/2020.

[WWA01] L. Wu, C. Weaver, and T. Austin. CryptoManiac: A fast flexible architecture for secure communication. In *International Symposium on Computer Architecture (ISCA)*, pages 110–119, 2001. https://doi.org/10.1109/ISCA.2001.937439.

[XLY+22] J. Xu, H. Lin, Z. Yuan, W. Shen, Y. Zhou, R. Chang, L. Wu, and K. Ren. RegVault: hardware assisted selective data randomization for operating system kernels. In *Design Automation Conference (DAC)*, pages 715–720, 2022. https://doi.org/10.1145/3489517.3530549.

[YHHF19] J. Yu, L. Hsiung, M. El Hajj, and C.W. Fletcher. Data oblivious ISA extensions for side channel-resistant and high performance computing. In *Network and Distributed System Security Symposium (NDSS)*, 2019. https://www.ndss-symposium.org/ndss-paper/data-oblivious-isa-extensions-for-side-channel-resistant-and-high-performance-computing.

[YHMH19] X. Yang, Y. Hou, J. Ma, and H. He. CDSP: A solution for privacy and security of multimedia information processing in industrial big data and Internet of Things. *Sensors*, 19(3):556:1–556:16, 2019. https://doi.org/10.3390/s19030556.

[YLW+25] Z. Ye, X. Li, C. Wang, R.C.C. Cheung, and K. Huang. RVSLH: Acceleration of postquantum standard SLH-DSA with customized RISC-V processor. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, pages 1–5, 2025. https://doi.org/10.1109/TVLSI.2025.3543352.

[YM04] P. Yu and T. Mitra. Scalable custom instructions identification for instruction-set extensible processors. In *Compilers, Architectures, and Synthesis for Embedded Systems (CASES)*, pages 69–78, 2004. https://doi.org/10.1145/1023833.1023844.

[YSKG14] K. Yumbul, E. Savaş, Ö. Kocabas, and J. Großschädl. Design and implementation of a versatile cryptographic unit for RISC processors. *Secure Communication Networks*, 7(1):36–52, 2014. https://doi.org/10.1002/sec.555.

[YSZ+24] Z. Ye, R. Song, H. Zhang, D. Chen, R.C.C. Cheung, and K. Huang. A highly-efficient lattice-based post-quantum cryptography processor for iot applications. *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2024(2):130–153, 2024. https://doi.org/10.46586/tches.v2024.i2.130-153.

[Yu96] A. Yu. The future of microprocessors. *IEEE Micro*, 16:46–53, 1996. https://doi.org/10.1109/40.546564.

[YYDZ08] X.-H. Yang, X.-R. Yu, Z.B. Dai, and Y.F. Zhang. Accelerated flexible processor architecture for crypto information. In *International Conference on Circuits and Systems for Communications (ICCSC)*, pages 628–632, 2008. https://doi.org/10.1109/ICCSC.2008.139.

[ZB19] F. Zaruba and L. Benini. The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(11):2629–2640, 2019. https://doi.org/10.1109/TVLSI.2019.2926114.

[ZKB+09] M. Zuluaga, T. Kluter, P. Brisk, N. Topham, and P. Ienne. Introducing control-flow inclusion to support pipelining in custom instruction set extensions. In *Symposium on Application Specific Processors (SASP)*, pages 114–121, 2009. https://doi.org/10.1109/SASP.2009.5226328.

[ZKGA20] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic. SonicBOOM: The 3rd generation Berkeley Out-of-Order Machine. In *Computer Architecture Research with RISC-V (CARRV)*, 2020. https://carrv.github.io/2020.

[ZQL+24] J. Zhou, G. Qin, L. Li, C. Guo, and W. Wang. ISA extensions of shuffling against side-channel attacks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 43(3):761–773, 2024. https://doi.org/10.1109/TCAD.2023.3323165.

[ZSM19] D. Zagieboylo, G. Suh, and A. Myers. Using information flow to design an ISA that controls timing channels. In *Computer Security Foundations Symposium (CSF)*, 2019. https://doi.ieeecomputersociety.org/10.1109/CSF.2019.00026.